



Lukas Kiem, BSc

Design and Implementation of Data Compression Methods for SoC-based Radar Systems

MASTER'S THESIS

to achieve the university degree of

Diplom-Ingenieur

Master's degree programme: Computer Science

submitted to

Graz University of Technology

Supervisor

Ass.Prof. Dipl.-Ing. Dr.techn. Christian Steger

Institute of Technical Informatics

Advisor

Dipl.-Ing. Heinz Neidhart (Infineon Technologies Austria AG)

Graz, May 2025

Kurzfassung

Die zunehmende Nachfrage nach sicheren und intelligenteren Fahrzeugen in der Automobilindustrie hat zu einer weit verbreiteten Einführung von Radarsensoren geführt, insbesondere in fortschrittlichen Fahrerassistenzsystemen und Anwendungen des autonomen Fahrens. Radarsensoren nehmen eine entscheidende Rolle ein, da sie in der Lage sind, selbst unter schwierigen Wetter- und Sichtbedingungen robuste und zuverlässige Umgebungsdaten zu liefern. Infolge der zunehmenden Komplexität radarbasierter Systeme sowie deren Integration in vielfältige Anwendungsbereiche, wie etwa die hochauflösende Kartierung, die Objektklassifizierung und die Mehrzielverfolgung, ist ein signifikanter Anstieg des Datenvolumens von Radarsensoren zu verzeichnen. Dies resultiert in Herausforderungen in Bezug auf Bandbreite und Speicherkapazität.

Diese Arbeit befasst sich mit diesen Herausforderungen durch die Entwicklung, Implementierung und Evaluierung eines effizienten Kompressionsschemas für Radardaten. Im Rahmen der Untersuchung wurden verschiedene Kompressionstechniken anhand ihrer Fähigkeit evaluiert, die Datengröße zu reduzieren, während gleichzeitig kritische Radarinformationen bewahrt werden. Die Implementierung und Evaluierung des finalen Algorithmus erfolgte auf einem AMD Zynq™ UltraScale+™ MPSoC.

Im Rahmen der Hardwareimplementierung wurde ein Evaluierungssystem realisiert, welches zentrale Komponenten, wie einen FFT IP Core, den Kompressions IP Core, einen AXI DMA IP Core für Hochgeschwindigkeits-Datenübertragungen und eine Bare-Metal-Anwendung zur Systemsteuerung beinhaltet. Das System wurde umfassend getestet und validiert. Dabei wurde ein verlustfreies Kompressionsverhältnis von 3,2 erreicht, wobei die Daten im 16-Bit-Festkommaformat vorliegen müssen. Es wurde eine Echtzeit-Performance demonstriert, die sich durch einen hohen Eingangsdatendurchsatz von 11,92 Gbit/s und eine geringe End-to-End-Latenz des gesamten Systems von lediglich 0,08 ms auszeichnet.

Abstract

The growing demand for safer and more intelligent vehicles in the automotive industry has driven the widespread adoption of radar sensors, particularly in advanced driver assistance systems (ADAS) and autonomous driving applications. Radar sensors play a pivotal role by providing robust and reliable environmental data even under harsh weather and visibility conditions. However, the volume of radar data has grown significantly with the increasing complexity of radar-based systems and their integration into applications like high-resolution mapping, object classification, and multi-target tracking. This data volume growth poses challenges in terms of bandwidth and storage capacity.

This thesis addresses these challenges by designing, implementing, and evaluating an efficient automotive radar data compression scheme. This work evaluates various compression techniques based on their ability to reduce data size while preserving critical radar information. The final algorithm was implemented and evaluated on an AMD Zynq[™] UltraScale+[™] MPSoC.

The hardware implementation is an evaluation system with key components such as an FFT IP core, the compression IP core, an AXI DMA IP core for high-speed data transfer, and a bare-metal application for system control. The system was extensively tested and validated, achieving a lossless compression ratio of 3.2 with the limitation that the data needs to be in a 16-bit fixed-point format. The system demonstrated real-time performance with an input throughput of 11.92 Gbit/s and an end-to-end latency for the whole system of 0.08 ms.

Acknowledgment

I would like to express my sincere gratitude to all those who have supported me throughout the completion of this master's thesis.

First and foremost, I would like to thank Ass.Prof. Dipl.-Ing. Dr.techn. Christian Steger for being my supervisor and providing invaluable guidance and support during this research.

I am also deeply grateful to Infineon Technologies AG for facilitating the cooperation that made this research possible. Special thanks go to Dipl.-Ing. Heinz Neidhart, for being an exceptional advisor and Dipl.-Ing. Dr.techn. Stephan Bernhart and Dipl.-Ing. Georg Schober for their professional expertise and assistance.

I would like to extend my heartfelt appreciation to my family for their unwavering support in various ways, my girlfriend for her constant encouragement, and my friends for their support throughout this journey.

Thank you all.

Graz, May 2025

Lukas Kiem

Contents

1	Introduction	11
1.1	Motivation	11
1.2	Objective	13
1.3	Outline	13
2	Background and State-of-the-Art	14
2.1	Automotive Radar	14
2.1.1	Introduction to Automotive Radar	14
2.1.2	Role of Radar in Modern Vehicles	15
2.1.3	CTRX8191F Radar Sensor	16
2.2	Radar Data Characteristics and Challenges	17
2.2.1	Characteristics of Radar Data	17
2.2.2	Radar Data Representation	19
2.2.3	Definitions	20
2.3	Data Compression for Automotive Applications	20
2.3.1	Introduction to Data Compression	20
2.3.2	Data Compression Process	21
2.3.3	Types of Data Compression	21
2.3.4	Requirements for Automotive Data Compression	22
2.4	Field-Programmable Gate Arrays (FPGAs)	23
2.4.1	Overview of the FPGA Technology	23
2.4.2	Advantages of FPGAs for Prototyping	23
2.4.3	Microcontroller Integration	24
2.5	Automotive System Architectures	24
2.5.1	Evolution of Automotive Architectures	24
2.5.2	Implications for Radar Data Management	26
2.5.3	Role of Data Compression in Modern Architectures	26
2.6	Radar Data Compression in Other Domains	26
2.7	Automotive Radar Data Compression	27
2.7.1	Texas Instruments (TI) Compression Engine	27
2.7.2	IFX Compression Engine	28
2.7.3	Doppler Redundancy Hybrid Encoding (DRHE)	29
2.7.4	LPC-Huffman Compression	31

3	Design	33
3.1	Design Requirements	33
3.2	MATLAB Simulation	34
3.2.1	Simulation Workflow	34
3.2.2	Input Data	36
3.3	Use Case and Requirements	42
3.4	Design Space Exploration	43
3.4.1	Design Space	43
3.4.2	Evaluation Metrics	44
3.4.3	Exploration Process	44
3.4.4	Exploration Results	45
3.4.5	Final Design Choices	45
3.5	Final Algorithm Design and Optimization	46
3.5.1	Remove Redundancy	46
3.5.2	Fixed Huffman Dictionary	49
3.5.3	Final Compression Scheme	49
3.5.4	Decompression Scheme	51
4	Implementation	53
4.1	Hardware Setup	53
4.2	Development Tools	55
4.2.1	Simulation, Testing, and Verification	56
4.3	Hardware Implementation Design	56
4.3.1	Bare Metal Application	57
4.3.2	AXI DMA IP Core	57
4.3.3	FFT IP Core	58
4.3.4	Compression IP Core	59
4.4	Challenges and Optimizations	62
4.4.1	Timing Closure	63
4.4.2	Fixed-Point Precision	63
4.4.3	Real-Time Performance Requirements	63
4.4.4	Resource Utilization	63
5	Results	64
5.1	Simulation Results and Analysis	64
5.2	Hardware Implementation: Results and Analysis	65
5.2.1	Evaluation Setup	65
5.2.2	Vivado Block Design	66
5.2.3	Processing Speed	66
5.2.4	Resource Utilization	69
5.2.5	Evaluation Against Simulation Results	70
5.3	Comparison to Requirements	72
6	Conclusion and Future Work	74
6.1	Future Work	75

A	Fixed Huffman Dictionary	76
B	Evaluation Hardware Setup	77
C	Acronyms	78
	Bibliography	81

List of Figures

1.1	Centralized architecture with SerDes	12
1.2	Centralized architecture with applied compression	13
2.1	The FMCW concept	15
2.2	A FMCW radar system	15
2.3	Virtual antenna array mapping	16
2.4	Sensor fusion with the ADAS applications	17
2.5	Pipeline of radar signal processing	18
2.6	Basic compression process	21
2.7	Modern hybrid zonal and central architecture	25
2.8	EGE Example with $k = 2$	28
2.9	Model predictions as IIR filters	30
2.10	DRHE encoding process	30
2.11	LPC-Huffman compression histogram comparison	32
3.1	Simulation workflow block diagram	34
3.2	Real-world data range FFT output	37
3.3	Real-world data: 3D range-Doppler image after NCI	37
3.4	Real-world data: 2D heatmaps at the different steps in the processing pipeline	38
3.5	Simulated data range FFT output	40
3.6	Simulated data: 3D range-Doppler image after NCI	40
3.7	Simulated data: 2D heatmaps at the different steps in the processing pipeline	41
3.8	Exploration process block diagram	44
3.9	Difference values histogram for the real part	46
3.10	Difference values histogram for the imaginary part	46
3.11	Optimal R4S4 symbol histogram	47
3.12	R4S4 symbol histogram	48
3.13	S4 symbol histogram	48
3.14	Fixed Huffman dictionary histogram	49
3.15	Top-level block diagram of the DRHE compression scheme	50
3.16	Block diagram of the DRHE model prediction block	50
3.17	DRHE APPEND-bits packing	51
3.18	Basic decompression scheme block diagram	52
3.19	Inverse model prediction for the decompression	52
4.1	Interface board used for the implementation	54
4.2	Simplified top-level system block diagram	57

4.3	Expected input data format by the FFT IP core	59
4.4	Compression pipeline with parallel processing	60
5.1	Scatter plot of the simulation results	65
5.2	Final Block Design in Vivado	66
5.3	Waveform showing the working principle of the symmetry splitter IP core .	67
5.4	Latency measurement of the compression IP core using the waveform viewer	68
5.5	Latency measurement of the FFT IP core using the waveform viewer	68
5.6	Implemented design in Vivado with highlighted leaf cells	71
B.1	Hardware evaluation setup with the used connections	77

List of Tables

2.1	Compression results	31
3.1	Configuration parameters for the real-world measurements	36
3.2	Configuration parameters for the simulated data generation	39
3.3	Example configuration Parameters for the ramp scenario	42
3.4	Design space definition	43
3.5	Design space exploration simulation results	45
3.6	Simulation results for DRHE without RLE	47
3.7	Mapping from the value range to the S4 value	51
4.1	AXI DMA configuration for the Read Channel	57
4.2	AXI DMA configuration for the Write Channel	58
4.3	FFT core configuration parameters	58
4.4	Configuration parameters and ranges for the FPGA implementation	59
5.1	Design space exploration results using the MATLAB simulation environment	64
5.2	Post-implementation resource utilization for the system	69
5.3	Post-implementation resource utilization for the compression IP core	69
5.4	Post-implementation resource utilization for the symmetry splitter IP core .	70
5.5	Design space exploration results: double precision vs. bit-accurate simulation	72
A.1	Fixed Huffman dictionary	76

Chapter 1

Introduction

With the rapidly evolving automotive industry and the shift towards electrification and automation, the amount of sensors used in a vehicle is growing. One sensor that plays a crucial role in this industry is the radar sensor. This type of sensor provides reliable information about the vehicle’s surroundings even under poor conditions. Radar is insensitive to poor visibility conditions such as rain, fog, or darkness, which makes radar sensors play a crucial role in ADAS and autonomous driving. At the beginning of the technology, the primary focus was on distance warning and crash avoidance. With increasing maturity, the applications became more complex and included adaptive cruise control (ACC), automatic emergency brake (AEB), blind spot detection (BSD), or lane change assist (LCA) [23]. Nowadays, the focus has shifted towards integrating radar sensors into more complex systems that support autonomous driving. These advanced applications include high-resolution mapping of the surroundings, object classification, and precise tracking of multiple targets, which are essential for enabling fully autonomous navigation and decision-making. The emphasis of all this development is to create a more intelligent, safer, and efficient driving experience.

Automotive Radar sensors are high-resolution and produce a vast amount of data, so they need significant bandwidth and storage capacity for transmission and processing. Efficient data compression techniques are needed to ensure systems can transmit and store the information. The design of the data compression techniques focuses on a high compression performance while not losing any critical details of the data.

1.1 Motivation

The motivation for this research originates from the essential role of radar sensors in automotive applications. As mentioned in Chapter 1, radar sensors are essential to various safety-critical functions. They are crucial to ADAS and autonomous driving mainly because of the redundancy they can provide to the system under harsh environmental conditions. This all-weather data availability makes radar sensors indispensable for these safety-first applications.

Automotive radar sensors generate a vast amount of data that needs to be transmitted, stored, and processed. This creates challenges related to storage capacity and bandwidth. With the shift from domain architecture to centralized and zonal architec-

ture, the amount of data that needs to be transmitted and stored gets even higher. In such architectures, resources are shared across the whole vehicle, which is divided into zones, and the data is aggregated in central controllers. This shift increases the need for efficient data compression solutions. Compressing the data at the sensor level minimizes the latency and helps the automotive industry reduce the resources needed to handle all the data. With that weight, costs, and complexity can be reduced when connecting all the resources. Figure 1.1 shows a block diagram of an example use case setup using a centralized architecture, without compression. Using the example configuration of 4 RX antennas, 50 Megasamples per second (MS/s) output sample rate, and a bitwidth of 12 bits per sample, the bitrate of 2.5 Gbit/s per corner radar is achieved. The front radar, for example, has a bitrate of 5 Gbit/s because of the 8 RX antennas. A SerDes system is needed to handle this large amount of uncompressed data. SerDes solutions are competing against other interconnection systems, like Ethernet. But they have the disadvantage of being a lot more expensive.

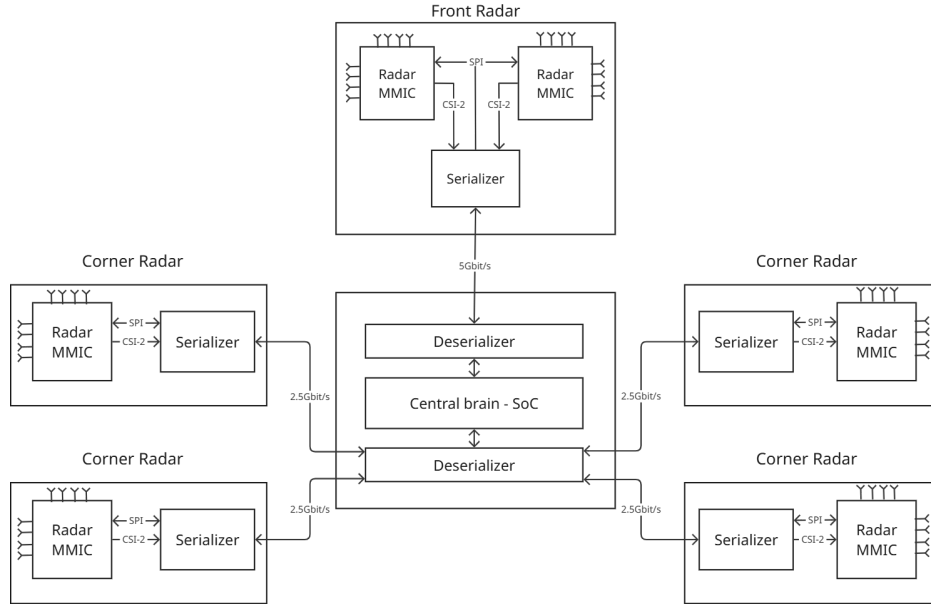


Figure 1.1: Centralized architecture with SerDes

In contrast, we can see in Figure 1.2 the same setup but with applied compression. In this specific case, compression is applied to the range-FFT data. In this example, the data is compressed from 2.5 Gbit/s to 1 Gbit/s, which a gigabit Ethernet cable can transmit. As a result, the system would be less expensive and perform at an equal level.

Field-Programmable Gate Arrays (FPGAs) offer a robust solution as a prototyping platform for developing hardware accelerators. Due to their inherent flexibility, parallel processing capabilities, and reconfigurability. In this thesis, the FPGA is a proof-of-concept platform to implement and evaluate the compression algorithm in a realistic automotive radar context.

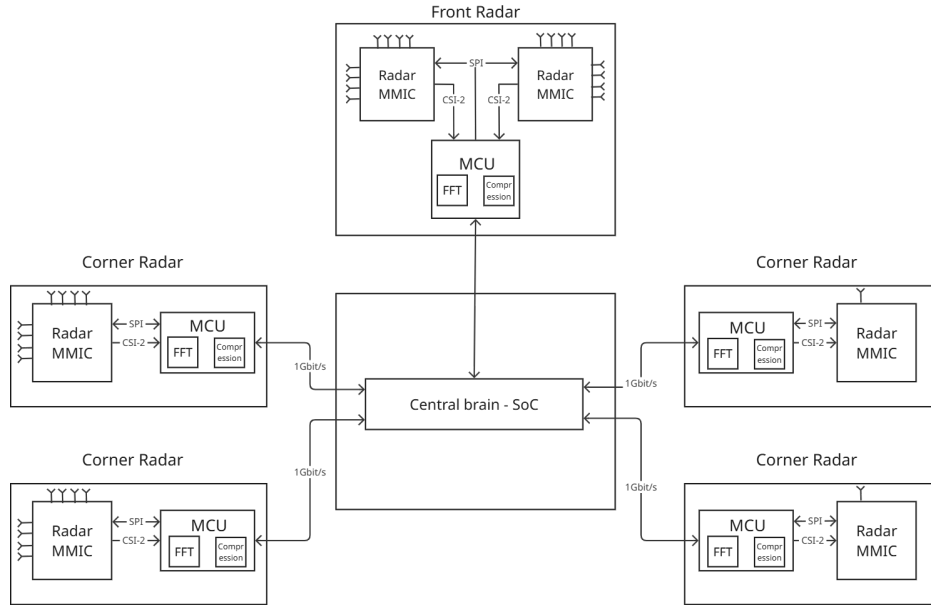


Figure 1.2: Centralized architecture with applied compression

1.2 Objective

The primary objective of this thesis is to develop and implement an efficient radar data compression algorithm for automotive applications. The algorithm is realized in a hardware implementation using FPGA technology. The goal is to achieve a real-time, resource-efficient compression solution that reduces data size while preserving critical information of the radar data. These goals are needed to meet the requirements of the automotive domain.

A MATLAB simulation environment supports the development and evaluation process. The environment tests and evaluates various compression algorithms regarding trade-offs between compression efficiency and data fidelity. Finally, the environment helps select and optimize the final solution. The ultimate aim is to implement the final algorithm on an FPGA platform, meeting the specified requirements.

1.3 Outline

Chapter 2 provides the needed background information for this work and gives an overview of the state of the art in radar data compression. Chapter 3 introduces the design of the algorithm, defines the design requirements, and introduces the MATLAB simulation environment. Furthermore, the design space exploration with the results and the final design choices is shown. Chapter 4 presents the hardware setup and final hardware implementation with the development flow. Additionally, the challenges and optimizations during the hardware implementation are shown. Chapter 5 first shows the simulation results and then the hardware implementation results and analysis. The hardware implementation results are then compared to the defined objectives of this work. Chapter 6 presents the conclusion of this thesis and proposes some possible future work.

Chapter 2

Background and State-of-the-Art

This chapter provides foundational knowledge and context. Beginning by introducing the fundamental principles of radar systems, their role in automotive applications, and the unique characteristics of radar data. The chapter then introduces data compression in general and defines the requirements for automotive data compression. As hardware implementation is central to this thesis, the chapter also discusses the advantages of FPGA technology for this work. Finally, the chapter explores state-of-the-art radar data compression.

2.1 Automotive Radar

2.1.1 Introduction to Automotive Radar

Radar systems have become a cornerstone of modern automotive technologies. They are essential to detect objects in the road environment, like other vehicles, pedestrians, or bicyclists. Automotive radar operates by emitting radio waves and analyzing the reflected signals from objects in the environment, which allows the system to accurately determine the distance, velocity, and angular position of surrounding objects. In automotive applications, the technology of frequency modulated continuous wave (FMCW) radar is typically used.

Figure 2.1 shows the basic FMCW concept. This FMCW radar concept works by emitting a continuous wave (CW) modulated in frequency. The signal is a linear chirp radiated toward a target through an antenna, and the echo received T_p short time later is mixed with a portion of the transmitted signal to produce a beat signal at a frequency f_b which is proportional to the round-trip time T_p .

Radar sensors offer the unique advantage of providing reliable data despite rough weather conditions such as rain, fog, snow, and darkness. Under such conditions, other sensors, such as cameras and lidar, fail to perform. This all-weather performance makes radar highly suited for automotive environments. Figure 2.2 shows a simplified FMCW radar system block diagram.

With the increasing demands of autonomous driving and ADAS, radar technology has advanced from single-input-multiple-output (SIMO) to multiple-input-multiple-output (MIMO) radar. SIMO radar consists of a single TX antenna and multiple output antennas. A SIMO's angle resolution depends on the number of RX antennas, so the number of RX

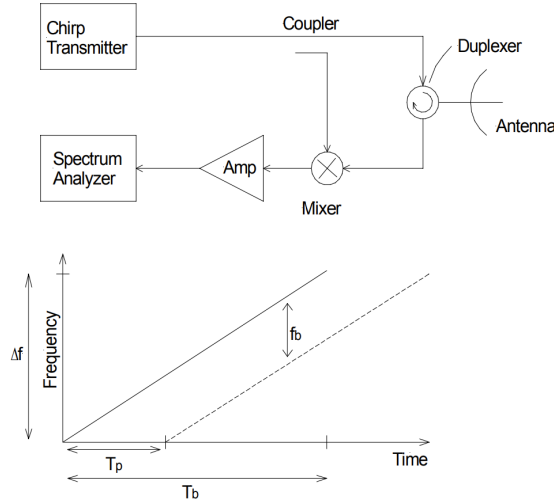


Figure 2.1: The FMCW concept [8]

antennas needs to be increased to improve the angle resolution. This approach has limits because each RX antenna needs its own RX processing chain on the device, which further connects to higher costs, higher power consumption, and more space. MIMO radar consists of multiple TX and RX antennas. It creates a virtual array of antennas by transmitting orthogonal, independent signals from each TX antenna and analyzing how the reflected signals are received across multiple RX antennas. Using the MIMO approach, the number of physical antennas can be reduced. As a result, the aperture size can be improved with fewer physical antennas. Imagine a SIMO radar system with 1 TX antenna and 12 RX antennas. To achieve the same aperture size with MIMO radar, just 4 RX and 3 RX antennas are needed. Figure 2.3 shows the mapping from physical antennas to the virtual antenna array.

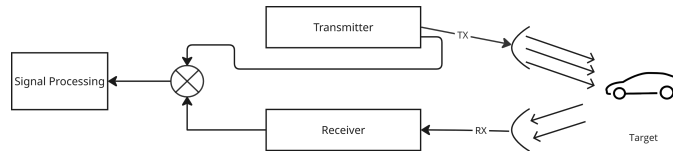


Figure 2.2: A FMCW radar system

2.1.2 Role of Radar in Modern Vehicles

Radar sensors play an essential role in the sensor network of a modern vehicle. An example of a sensor system with examples for the corresponding ADAS application can be seen in Figure 2.4.

Modern vehicles integrate data from various sensors to enhance their functionality. Radar plays a crucial role in this sensor fusion system due to its all-weather reliability. This means that radar can provide reliable data in challenging environmental conditions like fog, snow, rain, and darkness.

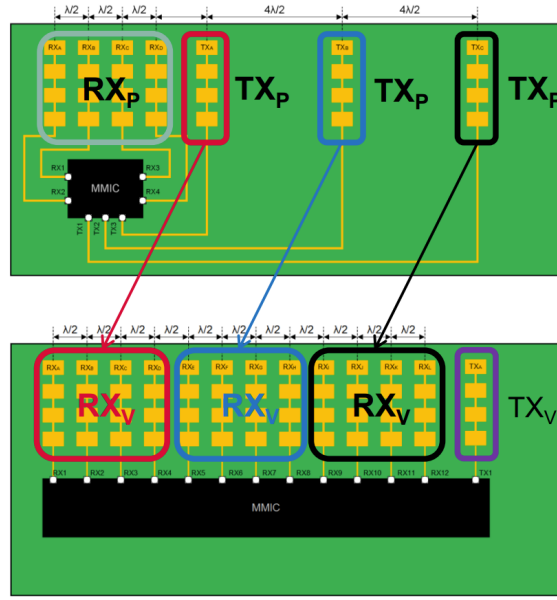


Figure 2.3: Virtual antenna array mapping

Additionally, advanced automotive radar employs FMCW technology at millimeter-wave frequencies. This allows for high-resolution target range and velocity estimation at a significantly lower cost than lidar technology. Furthermore, radar systems are effective for short- and long-range applications, making them suitable for detecting vehicles at long distances and assisting with parking and pedestrian detection in close proximity.

With its unique strengths, radar plays a significant role in modern vehicles. Moving towards L4/L5 autonomous driving capabilities, like all-weather reliability, is vital.

2.1.3 CTRX8191F Radar Sensor

The RASIC™ CTRX8191F is Infineon Technologies AG's (IFX) new 4D and imaging radar generation. It is a new state-of-the-art 28 nm radar monolithic microwave integrated circuit (MMIC) [31] of the Infineon XENSIV™ family [35]. It is designed to meet the requirements of SAE-defined L2+ to L4 autonomous driving. The CTRX8191F offers high performance at low system cost and a better signal-to-noise ratio than previous generations. This performance improvement is needed to detect road users and vehicles at a distance of up to 380 meters, using a system configuration with eight transmitters and eight receivers.

Furthermore, the CTRX8191F enables cascading of multiple devices at low frequencies. It contains a state-of-the-art digital PLL that offers high flexibility in generating complex waveforms with the shortest flyback time on the market ($<1 \mu s$).

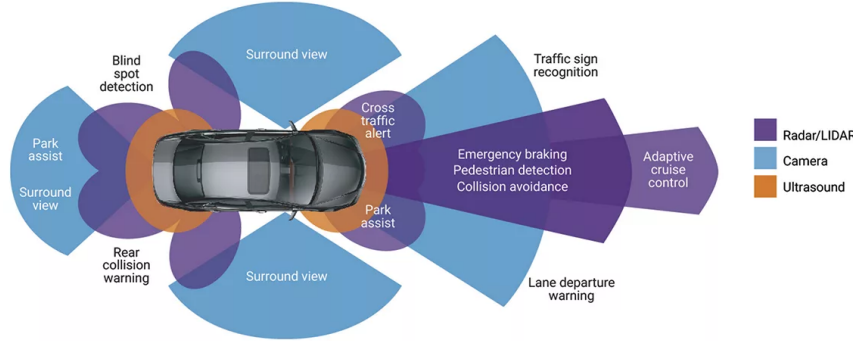


Figure 2.4: Sensor fusion with the ADAS applications [38]

2.2 Radar Data Characteristics and Challenges

2.2.1 Characteristics of Radar Data

Automotive radar sensors provide data that contains information about the surrounding environment. As introduced in Subsection 2.1.1 and shown in Figure 2.2, the signal reflected from the hit target and captured by the RX antenna is mixed with the TX signal by the mixer. The outcome of this process is the intermediate frequency (IF), which is then converted into digital values via the ADC. An automotive radar system typically works with multiple TX and RX antennas, resulting in multiple IF signals. All the information about the target that can be extracted, such as range, Doppler velocity, and azimuth angle, is contained in these IF signals [30]. Simplified, the range is determined by measuring the time it takes for the emitted signal to travel to the object and back. The velocity can be determined by analyzing the frequency shift in the reflected signal if the object moves relative to the radar sensor. This is called the Doppler effect. The azimuth angle can be determined using antenna arrays and beamforming techniques. In Figure 2.5, we can see the radar signal processing pipeline with a visual representation of the data structure. In Figure 2.5 (a), the frequency increases linearly over time, leading to frequency ramps, also called chirps. The ramp emitted by the TX antenna is shown in red, and the echo received by the RX antenna is shown in green. The IF signal is determined by mixing both signals, expressed by the variable f_b . Figure 2.5 (c) shows the samples acquired over a single ramp, and (d) shows the sample-ramp map, consisting of the samples over the configured number of ramps, which we call “frame” for convenience. Figure 2.5 (e) shows the frames over multiple RX antennas; we call this resulting 3D data structure “datacube”.

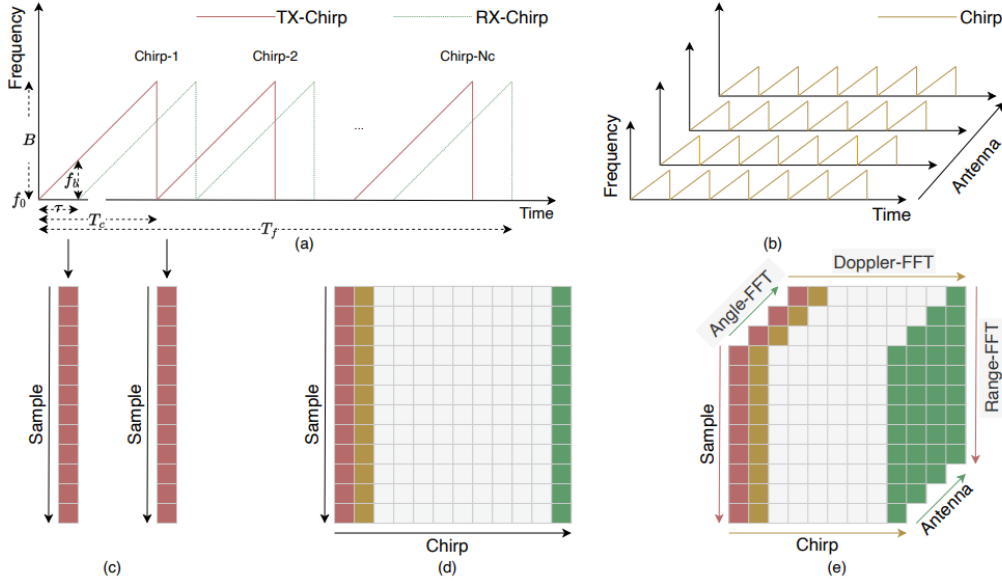


Figure 2.5: Pipeline of radar signal processing [30]. (a) Frequency ramps emitted by the TX antenna (red) and the echo received by the RX antenna (green), (b) Ramps received by multiple RX antennas, (c) Sampling performed over the samples of a ramp, (d) Sample-ramp map (frame), (e) One frame per RX antenna.

In the following part, we introduce how the information can be extracted from the raw IF data.

Range

By performing a fast Fourier transform (FFT) over the samples of one ramp (column), we get the IF signal in the frequency spectrum. Different peaks represent different objects at different ranges. As we can see in Figure 2.5, we call this operation “range-FFT”.

Velocity

To obtain the velocity of an object, multiple ramps need to be transmitted and processed. The ramps are transmitted separately at a specific time interval, denoted by T_c in Figure 2.5. Over multiple ramps, the peaks in the range-FFT spectrum stay the same but with different phases. Using this phase shift, the velocity can be obtained by performing a second FFT on the range-FFT, but this time in the Doppler direction (row), as shown in Figure 2.5. This step is called Doppler-FFT, and a 2D-FFT is performed with this step. Using this method, it is possible to separate multiple targets at the same distance but with different speeds. The outcome of this process is the range-Doppler Map.

Angle

Multiple RX antennas are needed to get the azimuth and elevation angle. With this method, targets at the same distance and with the same speed can be separated. A

third FFT is performed on the Doppler-FFT data in the RX antenna direction, called the Angle-FFT.

2.2.2 Radar Data Representation

Radar data can be represented in many ways during the different processing steps. In the following part, we will define the most relevant ones and define them [10].

Raw Data

The unprocessed or minimally processed data that comes directly from the radar sensor. We call it “raw” ADC data. Storing the raw data for all samples, ramps, and RX antennas results in a data cube.

Range Map

In the range-FFT, the raw ADC signal is converted to a representation in the frequency domain. Storing it over the ramps results in a range-ramp map, as shown in Figure 2.5 (d). One map for each RX antenna results in a 3D range cube.

Range-Doppler Map

After performing the Doppler-FFT over each range bin, the output is a range-Doppler map. This format represents the radar data as a 2D map of range vs. velocity. Over multiple RX antennas, it results in a 3D range-Doppler cube.

Detection List

After further processing, which we call 2nd stage processing, the outcome is the detection list. It is a high-level data format containing a condensed list of detected objects, each tagged with parameters like position, velocity, and size. Application examples are ACC, collision avoidance, and automated parking.

Point Cloud

This is another high-level data format with individual data points representing the signal reflections. Point clouds are typically represented in sensor fusion systems to build a detailed view of the environment.

The data is smaller in size in the detection list or point cloud format, and it is easier to transmit and store it in such a format. However, the problem is that the whole system loses flexibility because important radar data is discarded during the processing. In a modern vehicle with a centralized architecture, the processing system and the sensor fusion algorithm may need all the data, and there is no way to recover this discarded data. Thus, using high-level processed data formats is no option. So, the aim is to transmit raw data, range-FFT result, or Doppler-FFT result.

2.2.3 Definitions

Noise floor (NF) is the level of unwanted background noise present in the data. It is the sum of all noise sources within the radar system. It is calculated by taking the root mean square (RMS) of the noise in dB

$$NF = rms(N_{dB}) \quad (2.1)$$

Signal-to-noise ratio (SNR) is the ratio of signal power to noise power. Typically, it is expressed in decibels (dB). A ratio higher than 0 dB indicates more signal than noise.

$$SNR = \frac{P_{signal}}{P_{noise}} \quad (2.2)$$

where P is the RMS power [4]. If the signal and noise measurements are already in decibel form, the noise can be subtracted from the signal

$$SNR = S_{dB} - N_{dB} \quad (2.3)$$

where S_{dB} is the RMS signal measured in dB and N_{dB} is the RMS noise measured in dB.

2.3 Data Compression for Automotive Applications

2.3.1 Introduction to Data Compression

Data compression is the process by which information is encoded into fewer bits than the original representation. Data compression, source coding, or bit-rate reduction [11] is split into two major categories: lossy or lossless compression.

Lossy compression reduces bits by identifying and removing less important information. This removed information is lost and cannot be restored.

Lossless compression reduces bits by identifying and exploiting correlation and redundancies in the data. The compression is achieved by exploiting correlations and/or removing these redundancies, while no information is lost using this approach.

Typically, lossy techniques provide higher compression ratios than lossless techniques. Whether the loss of information in lossy techniques is acceptable depends on the field of application. In safety-critical applications, like adaptive cruise control, less loss is acceptable during compression than in audio or image compression.

Data compression is needed in many applications where lots of data is transmitted and/or stored. Data compression helps reduce these storage and transmission bandwidth needs. In return for the reduced size, computational complexity is introduced for compression and decompression. As a result, a space-time complexity trade-off is created.

To express the performance of a compression algorithm in reducing the size, the compression ratio (CR) metric is used, which is defined as follows:

$$CR = \frac{\text{Number of input bits}}{\text{Number of output bits}} \quad (2.4)$$

2.3.2 Data Compression Process

As introduced in Section 2.3.1, data compression techniques help reduce the needed transmission bandwidth and storage resources. A basic compression process is shown in Figure 2.6. The original data runs through the compression block (compressor or encoder); the result is the compressed data, which is smaller in size. The compressed data can be transmitted and stored more efficiently than the original uncompressed data. The compressed data is decompressed through the decompression block (decompressor or decoder).

In lossy compression techniques, the outcome after decompression does not exactly match the original data. In the case of lossless compression, it exactly matches the original data.

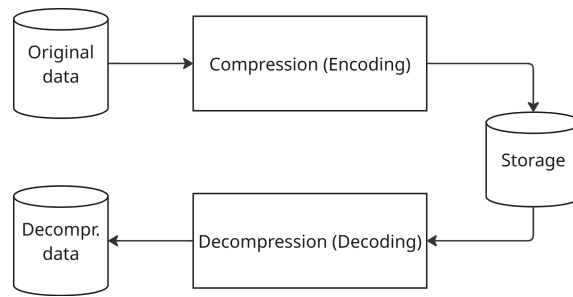


Figure 2.6: Basic compression process

2.3.3 Types of Data Compression

There are various types of data compression techniques. The following section introduces the major techniques with respect to whether they are lossy or lossless and the field of use [37, 16, 14].

Entropy-Based Methods

Any lossless scheme based on entropy coding uses statistical models of the input data to encode frequently occurring symbols with shorter codes and less frequent symbols with longer codes. Such techniques typically achieve moderate compression ratios without losing information. The compression ratio can be further increased by quantizing the input data to lower precision and applying the statistical model to the quantized data. Popular entropy coding techniques are Huffman coding [1] and arithmetic coding [3].

Huffman Coding

Huffman coding is based on entropy coding and is used for many data types, such as text, audio, video, and image. It is a lossless technique, and the basic idea is to use a variable-length code to encode a source symbol. The variable-length code is derived based on the estimated or calculated probability of occurrence for each possible value of the source symbols. After calculating the probability, a complete binary tree is built in descending order, and in this tree, high-frequency symbols are assigned codes with fewer bits. Less

frequent symbols have code with more bits. The outcome is a table with a 1:1 mapping from a symbol to a code and vice-versa [1, 7].

Run-Length Encoding (RLE)

RLE is a simple and popular compression technique. It is used on data with consecutive repeated values, and this sequence is then represented with a shorter sequence. For example, consider data with 15 consecutive 'A's. Using RLE, this sequence can be compressed to '15A', which needs two bytes instead of 15 bytes. Typically, it can be used in data with high redundancy or as a combination with other compression techniques.

Predictive Coding

Predictive coding is based on predicting the next value in a sequence. The sequence must have a correlation that can be used for prediction. The prediction value is subtracted from the real value, and the outcome should ideally be small values, which can be compressed using another technique, such as Huffman coding. Predictive coding is lossless in its simplest form, as the difference, also called delta, can have a high precision to restore exactly the original data. However, in practice, the differences are quantized to achieve higher compression ratios. It can be used on any data with a correlation present, such as radar data with spatial or temporal correlation.

Transform-Based Methods

Transform-based techniques are used on natural data like audio signals or images. This technique is typically lossy. Using transformation techniques like fast Fourier transform (FFT), discrete cosine transform (DCT), and discrete wavelet transform (DWT), the data is transformed into a form where it is easier to choose which information to discard. This is an efficient and helpful technique for applications with tolerably small data losses. These transform-based techniques also possibly show correlation in the data that can be used for compression.

2.3.4 Requirements for Automotive Data Compression

Data compression in the automotive context has to fulfill some additional requirements. Since the compression technique is used in safety-critical applications, the data compression algorithm has to be low-latency to meet the real-time requirement, provide high reliability, and preserve critical information. In the best case, it should be lossless while delivering a high compression ratio. In the following part, we present each requirement in more detail.

Real-Time

In automotive safety-critical applications, the compression algorithm must operate with minimal latency and provide a high throughput to ensure real-time performance. The algorithm must be capable of handling high data rates and must be computationally efficient.

Preserve Information

The algorithm must preserve the critical information of the radar data. Any loss of data fidelity can and will compromise object detection, tracking, and classification. The best case would be lossless compression, but lossy techniques are also acceptable if the critical features are preserved.

Low Computational Complexity

As automotive radar systems typically operate on hardware with limited resources, the algorithm must be lightweight enough to run on such embedded systems. The algorithm should minimize resource usage to fit within the constraints of embedded platforms.

High Reliability

Radar systems are integral to safety-critical applications, and any failure in the compression or decompression algorithm could compromise safety. Robust error-handling techniques must be employed in the system.

2.4 Field-Programmable Gate Arrays (FPGAs)

2.4.1 Overview of the FPGA Technology

An FPGA is a hardware device where the hardware logic can be programmed and reprogrammed. FPGAs consist of a matrix of configurable logic blocks (CLB) linked to each other by an entirely reprogrammable interconnection network [9]. This allows designers to create custom digital circuits. These capabilities are ideal for developing custom, high-performance applications where parallel processing and real-time performance are critical.

2.4.2 Advantages of FPGAs for Prototyping

FPGAs are an ideal choice for the final radar data compression implementation of this work because they provide several advantages. The following part will list some of the key advantages.

Reconfigurability

FPGAs' primary advantage is their reprogramming ability, allowing for an iterative design and testing approach. FPGAs can be programmed and reprogrammed using a hardware description language (HDL), such as VHDL or Verilog.

Parallel Processing

FPGAs enable multiple operations to run in parallel. This is ideal for processing large volumes of data, like in a radar sensor application. Unlike traditional processors, which rely on sequential execution, FPGAs can accelerate compute-intensive tasks by distributing them across multiple hardware blocks that run in parallel, making them suitable for developing a real-time performance algorithm.

Hardware-Software Integration

FPGAs provide tight integration between programmable logic (PL) and the processing system (PS), allowing seamless communication between hardware accelerators and software. This makes them ideal for prototyping systems requiring interaction between hardware and software components.

Rapid Prototyping

FPGAs are ideal for prototyping. Complex techniques like radar data compression need extensive testing and multiple adaptations during the development process. With FPGAs, this is perfectly possible before finalizing a production design. This enables higher cost-effectiveness for the development process compared to the development of application-specific integrated circuits (ASIC).

2.4.3 Microcontroller Integration

As introduced in the Subsections 2.4.1 and 2.4.2, FPGAs are ideal for this work. With the use of FPGAs, it is possible to reduce the time-to-market since it is possible to develop, test, verify, and optimize designs quickly. Finally, integrating an FPGA into the system would be too costly if the design is to be used in an automotive application. The final design can be transitioned to an ASIC for mass production. ASICs provide higher performance, lower power consumption, and reduced cost than FPGAs. These properties are ideal for large-scale deployment in vehicles. Another option is integrating the compression core and the needed pre- and post-processing steps into a microcontroller or System-on-Chip (SoC). The blocks can be integrated as hardware accelerators.

2.5 Automotive System Architectures

2.5.1 Evolution of Automotive Architectures

Automotive system architectures have significantly evolved over the decades. Driven by the increasing demand for safety, functionality, efficiency, and adaptability, vehicles have transformed from simple mechanical systems into highly complex, software-driven machines. Electronics and software had to adapt to growing needs continuously, so architecture had to adapt from simple standalone systems to domain-based designs, modern, software-centric approaches like zonal and central architectures, and the software-defined vehicle (SDV) [26]. The following subsections explain the different architectures in more detail.

Standalone System

In the early stages of automotive electronics, primarily mechanical and analog components handled individual functions. Electronic components, like the anti-lock braking system (ABS), were slowly introduced to handle specific tasks.

Domain-Based Architecture

The domain-based approach was born to handle a vehicle's growing functionality and complexity. The architecture groups related functions into domains, and each domain is controlled by a domain controller, a set of electronic control units (ECU). Examples of domains are powertrain, chassis, and infotainment [32].

Centralized Architecture

With the advancements in connectivity, autonomous driving, and more complex ADAS applications, the centralized architecture was developed. This approach uses fewer but more powerful components, called central “brains”, where data from many sensors is collected. This enables the sensor fusion approach, where data from multiple sensors is combined and processed to achieve higher detection resolution and better classification performance of the environment. However, with this approach, several challenges arise. It requires high bandwidth to handle all the data streams from different components, like radar, lidar, and cameras, across the whole vehicle [20].

Zonal Architecture

The zonal approach divides the vehicle into geographical zones to simplify wiring. Each zone has a zonal controller responsible for managing all the sensors, actuators, and communication within that zone. This approach emerged in parallel with the centralized approach. The zone controllers are typically used for low-level processing and as zonal gateways for transmission to the central brain [28].

Software-Defined Vehicle

This is the latest evolution. It shifts the design focus from hardware to software. In this approach, most of the vehicle functionality is driven by software. It uses a hybrid zonal-centralized architecture where zone controllers perform the low-level processing and act as zonal gateways, and the central microcontroller unit (MCU) performs the complex and resource-intensive tasks, such as sensor fusion processing and further processing. Figure 2.7 shows a hybrid zonal and central architecture.

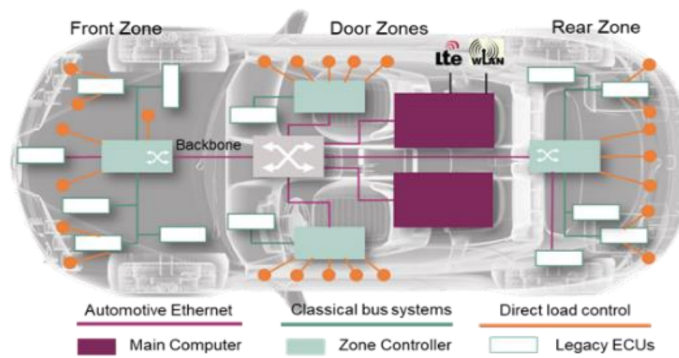


Figure 2.7: Modern hybrid zonal and central architecture [33]

2.5.2 Implications for Radar Data Management

With the shift towards software-defined vehicles with centralized computing using a central brain, the system is more flexible compared to other architecture approaches, and software can be maintained more easily since changes have to be made in a single place. Additionally, using this approach, the system is more scalable for changes in the demand for functionality.

On the other hand, there are negative implications, or better said, challenges that still need to be solved. To keep the applications as flexible as possible, the sensor data, in general, should be transmitted as raw as possible. This means the sensor data should not be processed much before being transmitted to the central MCU. Since a radar sensor produces a large amount of data, the bottleneck moves to the transmission bandwidth. There are two possible solutions: Choosing a different transmission solution that can handle larger amounts of data or reducing the data size. Different transmission solutions that can handle a larger amount of data are already available, but are more expensive or not mature enough for use. This work focuses on the latter solution, reducing the transmitted data size using data compression techniques.

2.5.3 Role of Data Compression in Modern Architectures

Modern automotive architectures, such as zonal and centralized architectures, have evolved to address ADAS's and autonomous driving's growing complexity and data demands. As introduced in Subsection 2.5.1, these architectures use sensor fusion to improve the resolution and detection performance. Sensor fusion relies on large volumes of sensor data, introducing significant bandwidth and storage challenges. Data compression plays a critical role in overcoming these challenges. It enables efficient data handling while maintaining system performance and ensuring the reliability of safety-critical applications. In such modern automotive architectures, the compression happens on the zonal controllers to compress near the sensor. This helps meet the real-time requirements. The compressed data is transmitted to the central controller and stored in memory. The central controller performs the decompression and further processing. This work focuses on reducing the needed bandwidth by reducing the data size.

2.6 Radar Data Compression in Other Domains

There are many works about radar data compression in general. So, by researching radar data compression without the automotive context, we can find several works that have been done with a focus on other domains, such as USV and weather radar. The work [24] presents a compression method for USV radar data. The method meets both real-time and lossless requirements, and it operates on radar video that has undergone range quantization, amplitude quantization, and azimuth quantization. It is based on combining the VarByte algorithm and RLE compression, achieving remarkable compression ratios.

Furthermore, some works on weather radar data have been found, such as [17, 18]. The work [17] proposes a hybrid combination of RLE, a lossless, and a lossy method. The method is developed to compress the processed 8-bit weather radar data by exploiting the data redundancies, such as many consecutive zeros. The solution in [18] also works on high-

level processed data. Using spatial and temporal prediction, it exploits the three weather radar data characteristics: sparsity, spatial redundancy, and temporal redundancy. The prediction values are subtracted from the real value, and the resulting residuals are then encoded using entropy coding. In the work [22], the authors propose a compression solution for radar images, and in [5], several algorithms for synthetic aperture radar (SAR) raw data compression are compared.

These methods are not directly suitable for automotive needs. Either they are lacking in fulfilling the real-time requirement, exploiting data characteristics not present in automotive FMCW radar data, or compressing high-level processed radar data. Some of the approaches may be useful for this work.

2.7 Automotive Radar Data Compression

With the knowledge from the research about radar data compression in general, from Subsection 2.6, in this subsection, we dive into the research about state-of-the-art automotive radar data compression methods, presenting the methods from different works and also various vendors.

2.7.1 Texas Instruments (TI) Compression Engine

In the white paper [19], the authors introduce the so-called TI Memory Compression/Decompression Engine from TI. This engine is a newly added hardware module to the TI millimeter wave radar solutions.

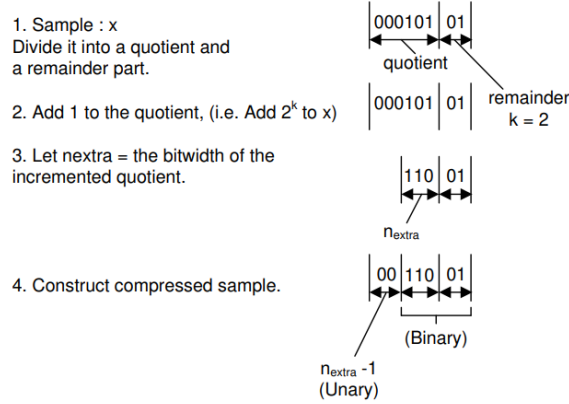
The method is developed for FMCW radar to virtually increase the amount of data stored in the onboard RAM. The engine uses exponential Golomb encoding (EGE) [2] to exploit sparsity in the range dimension and an optimization step that selects the best Golomb parameter based on the most common bit-width. The compression engine is designed to achieve an arbitrary compression ratio, which means lower compression ratios save more memory but result in more quantization loss.

The compression algorithm is an order- k exponential Golomb encoder, which encodes each sample to occupy a space approximately proportional to its bit-width. The algorithm is described in [2]. Order- k EGE is parameterized by a nonnegative integer parameter k , the most common bitwidth in the input vector, to control the trade-off between code length and encoding complexity. This means it is the boundary line between the variable-bitwidth quotient and fixed-bitwidth remainder parts. In Figure 2.8, an example is shown where the number 23 is encoded using EGE and $k = 2$.

The compression algorithm was tested on field data in the range and antenna dimensions. The following two subsections show the test results in both directions.

Results in the Range Dimension

8 range bins were grouped into a block and compressed in the test. With a compression of 40 % (CR = 2.5) of the original range-FFT radar data, the results are nearly indistinguishable from the original data. Using a compression of 33 % of the original size, ridges are seen across the Doppler dimension, especially where strong objects are located. The results also show that the higher the CR, the more the noise floor is raised. Compression

Figure 2.8: EGE Example with $k=2$

rates of 33 % or more should not affect detection performance if the detection algorithms can ignore these ridges.

Results in the Antenna Dimension

In this test, 4 complex samples collected over 4 RX antennas for a single range bin are grouped as a block and compressed. Again, the results are similar: the performance is relatively good at a compression of 40 % ($CR = 2.5$). However, with a compression of 33 %, more dominant ridges can be seen than in the compression over the range dimension. For practical use, the authors recommend using a compression of 62.5 % ($CR = 1.6$) if no distortion is desired.

2.7.2 IFX Compression Engine

Multiple selectable solutions to compress FMCW radar data are embedded into the newest microcontroller family by Infineon Technologies AG, the AURIX™ TC4xx family. In the following part, we will introduce the three solutions to reduce the size of the radar data.

Reduced Precision

This is the simplest form of reducing the size, simply reducing the precision from 32-bit to 16-bit precision. This method achieves a CR of 2 (50 %) compared to 32-bit data.

CMPRA

This method is a common exponent compression. It packs 16 complex bins into a 256-bit packet. Each 256-bit container represents 16 pairs of real and imaginary data, stored with a common exponent and separate mantissa values for each data value. 16 complex bins with 32-bit precision would occupy $16 \times 32 \times 2 = 1024$ bits without compression. That means we have a CR of 4 (25 %) compared to 32-bit precision. Compared to bins with 16-bit precision, we achieve a CR of 2 using this method.

CMPRB

This method is also a common exponent compression. It achieves to pack 32 complex bins into a 256-bit packet. Each 256-bit container represents then 32 pairs of real and imaginary data, stored with a common exponent and separate mantissa values for each data value. The resulting CR is 8 (12.5 %) compared to samples with 32-bit precision, and the CR is 4 compared with 16-bit precision.

By testing the three methods on a radar dataset and comparing them against the 16-bit floating point variant using CMPRA, all targets can be detected, and no false positive detections are caused. Additionally, the noise floor is raised from approximately -90 dB to -70 dB. Using CMPRB, there are 3.27 % false negative target detections and 0.44 % false positive detections. The noise floor is raised to -52 dB, so significantly more. So, there is a trade-off between high CR and high-precision detection.

2.7.3 Doppler Redundancy Hybrid Encoding (DRHE)

The DRHE method is presented in the work [12] and in the IEEE paper with the title “On-the-fly Range-Velocity Radar Data Compression Exploring Doppler Spectrum Redundancy” [13]. It is an FMCW radar data compression algorithm that works on-the-fly. On-the-fly means that the data can be processed and compressed immediately as received from the radar sensor with no or minimal buffering. The scheme exploits the correlation of the radar data after the range-FFT in the Doppler dimension by performing model prediction over the Doppler dimension. The prediction is done in the polar form using two IIR filters with tunable parameters, α and β . The prediction formula is:

$$\hat{X}[m, n] = \hat{a}[m, n]e^{j\hat{\theta}} \quad (2.5)$$

where $\hat{a}$ and $\hat{\theta}$ are the predicted magnitude and phase. The magnitude prediction is defined as:

$$\hat{a}[m, n] = \alpha\hat{a}[m - 1, n] + (1 - \alpha)a[m - 1, n] \quad (2.6)$$

The phase prediction is defined as:

$$\begin{aligned} \hat{\theta}[m, n] &= \theta_{mdl}[m, n] + \theta_{incMdl}[m, n] \\ \theta_{mdl}[m, n] &= \theta[m - 1, n] \\ \theta_{incMdl}[m, n] &= \beta\theta_{incMdl}[m - 1, n] + (1 - \beta)(\theta[m - 1, n] - \theta_{mdl}[m - 1, n]) \end{aligned} \quad (2.7)$$

If we substitute θ_{mdl} and θ_{incMdl} into $\hat{\theta}$ we get:

$$\beta\hat{\theta}[m - 1, n] + (2 - \beta)\theta[m - 1, n] - \theta[m - 2, n] \quad (2.8)$$

The prediction filters as a data flow graph are shown in Figure 2.9.

After calculating the prediction results, the difference between the measured data and the prediction is calculated using:

$$X[m, n] - \hat{X}[m, n] \quad (2.9)$$

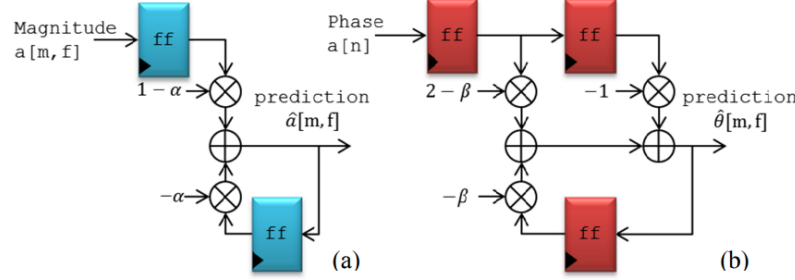


Figure 2.9: Predictions as IIR filters: (a) Magnitude prediction (b) Phase prediction [12]

The resulting difference is then further encoded by the encoding process, as shown in Figure 2.10. One characteristic of the difference data is that there are many zeros in the data present, which are encoded (a maximum of 15 zeros) into 4 bits, called the RRRR part. Then, in the SSSS part, the needed number of bits for the APPEND part are encoded. The APPEND part is the part that contains the difference value. So, the APPEND bits are of variable length, but the R4S4 part is fixed to 8 bits. These 8 bits can be encoded using Huffman coding. In the work, they are using a fixed Huffman dictionary. Additionally, they proposed an implementation for efficient memory addressing, which could cause data loss because of a fixed compression ratio.

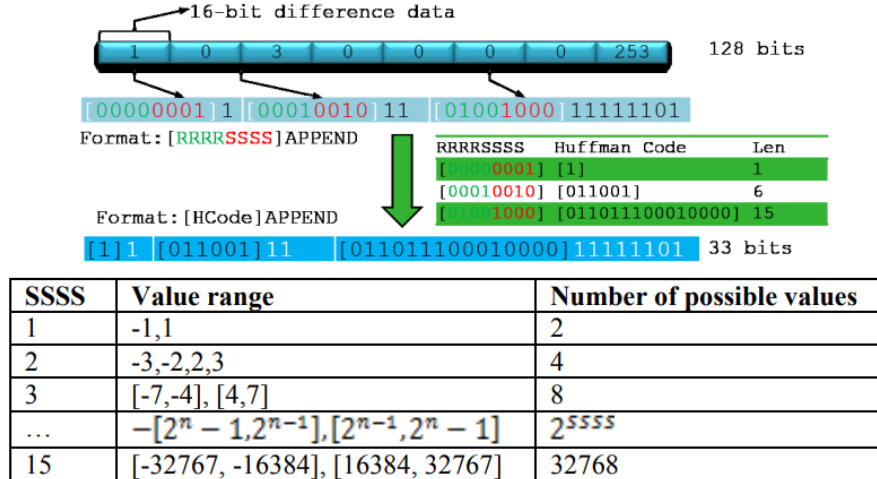


Figure 2.10: Encoding process [13]

The authors tested the algorithm with simple simulated data with 10 randomly placed targets with random speeds, complex simulated data with multiple targets at the same distance with different speeds, and real data collected on a street. The algorithm achieved some remarkable results in these tests.

In Table 2.1, the results of the test runs are shown. The “Ideal” row was performed without the practical memory addressing solution, and the achieved compression ratios are remarkably high for a lossless solution. In the rows for the “practical” solution in the bracket, the fixed compression ratio is given, and as a result, the percentage of data loss.

	Simulated data (simple)	Simulated data (hard)	Real data
Ideal	9.8	7.3	9.7
Practical (CR=5) - (% data loss)	0 %	0.04 %	0 %
Practical (CR=6) - (% data loss)	0 %	1.8 %	0.005 %
Practical (CR=7) - (% data loss)	0.005 %	3.0 %	0.42 %

Table 2.1: Compression results

2.7.4 LPC-Huffman Compression

In the work [25], the authors present a solution to compress FMCW radar data using a combination of linear predictive coding (LPC) and Huffman coding. The input data is said to be `int16` data. Therefore, we assume that this work is performed on raw ADC data. In the following part, we break down both steps of the algorithm.

LPC is a method used to predict the value of the signal at a given time based on past values. The goal is to minimize the error between the predicted and real measured values. The prediction is defined as:

$$\tilde{s}_n = - \sum_{k=1}^p a_k s_{n-k} \quad (2.10)$$

So, the aim is to predict s_n using the last p samples. After that, the error between the real value s_n and the predicted value $\tilde{s}_n$ is calculated. It is given by:

$$e_n = s_n - \tilde{s}_n = s_n + \sum_{k=1}^p a_k s_{n-k} \quad (2.11)$$

The error is also called the residual. The least squares error approach is used to optimize the predictor coefficient a_k .

Huffman coding encodes the low-entropy residuals into fewer bits. The LPC algorithm reduces the signal's dynamic range to a few coefficients around the zero value. A comparison of the value range can be seen in Figure 2.11. Knowing this distribution of difference values, Huffman coding is a good choice for assigning shorter codes to more frequent values. In this work, the Huffman dictionary is computed for input data, unlike in 2.7.3 where a fixed Huffman dictionary was used.

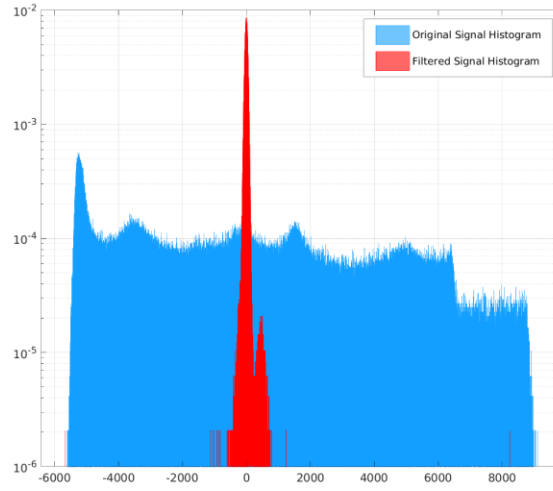


Figure 2.11: Histogram comparison (red) Residuals after LPC (blue) Original input values [25]

The test results in the work achieve a CR of 2.056.

Chapter 3

Design

This chapter focuses on developing an efficient radar data compression algorithm that meets the strict requirements of automotive radar applications. First, the requirements are defined in Section 3.1, then the simulation environment is introduced in Section 3.2, and a use case is defined in Section 3.3. Finally, the design space is defined, explored, and based on the results, a final decision on which approach to take is made in Sections 3.4 and 3.5.

3.1 Design Requirements

The design of the radar data compression algorithm is guided by requirements that need to be fulfilled in the automotive domain. These requirements ensure the compression algorithm is compatible with the safety-critical ADAS applications and scalable for different use cases and future radar systems. The four main requirements we focus on in this work are explained in the following subsections.

Real-Time Performance

The system must operate with minimal latency to process radar data in real-time. The data rates produced by a radar sensor are high, and the algorithm must be able to process this amount of data. Any significant delay in compressing or decompressing could impact the system's ability to make timely decisions.

Data Fidelity

The compression algorithm must preserve all critical radar information. Since the critical features are not identified at that stage, all the data must be maintained for further processing.

High Compression Efficiency

The compression algorithm must achieve a high compression ratio without losing critical information.

Scalability

The algorithm should support different radar resolutions. That means the number of RX antennas, the number of ramps, and the number of samples should be configurable.

The tradeoff between data fidelity and high compression efficiency has to be handled. High data fidelity typically means a lower compression ratio, and high compression ratio typically means higher loss of information.

3.2 MATLAB Simulation

The purpose of this section is to describe the MATLAB-based simulation environment. MATLAB was chosen because of its rich set of tools and functionality. It is perfectly suitable for signal processing, algorithm prototyping, and data analysis. The modular approach of the simulation environment allowed us to rapidly develop different compression techniques and evaluate each component in the processing chain separately.

3.2.1 Simulation Workflow

For simulating various compression techniques to evaluate the performance using different metrics, MATLAB 2023b [29] is used. The environment was developed using a modular approach where each component of the compression processing pipeline can be developed and evaluated independently. For the simulation, we used double precision, and if needed, used quantization to reduce the precision. Since only one state-of-the-art work, the “LPC-Huffman compression” introduced in Subsection 2.7.4, is based on compressing the raw ADC data, we shifted the focus to compressing the range FFT data. The FFT introduces correlation and sparsity into the data because typically only a few frequency bins contain significant energy, corresponding to the reflections from objects, while the rest contain smaller values, corresponding to the noise. Furthermore, the peak where an object is located is predictable over the Doppler dimension as shown in work [12]. Figure 3.1 shows the workflow as a block diagram. The following subsections describe each step in detail.

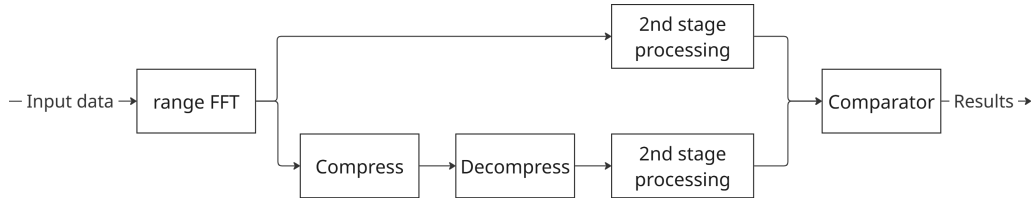


Figure 3.1: Simulation workflow block diagram

Input Data

The raw ADC data is loaded from a file. There are different bit widths of raw data available, which we reference as *bitWidth* in this work. The data is structured in a way that would be received directly from the sensor. The data we use in the simulation is described in 3.2.2.

Preprocessing

In this step, the data is prepared for further processing to increase the accuracy and use the full available precision. The preprocessing consists of two steps:

1. **Shift to full-scale** - The raw ADC data is shifted to 32-bit full-scale as a first step. That means it is shifted by $16 - \text{bitWidth} + 16$ bits to the left, where *bitWidth* is determined by the ADC data format.
2. **Windowing** - To reduce the effects of spectral leakage and improve the radar data's frequency resolution, the data is multiplied by a window. We use a Hanning window with a length equal to the number of samples.

Range FFT

After the preprocessing, the FFT is performed. As described in 2.2, we extract the range information about the targets from the raw ADC data with the range FFT.

Range FFT - Postprocessing

The purpose of this step is to prepare the data for compression and the upcoming processing steps. There are two major steps performed:

1. **Scale the range FFT output** - The output is divided by the number of samples. This step ensures that the signal's energy remains consistent after the FFT, so this is needed in the hardware implementation to prevent overflow in the FFT core.
2. **Take symmetric first half** - Since we input purely real-valued data to the FFT, the output consists of two symmetric halves [6]. So, we keep the first half of the elements.

Compress

This step is the central part of this work, where the different compression techniques are performed. It includes all the necessary steps, such as quantization, to reduce the precision and entropy coding.

Decompress

The data is reconstructed from the compressed representation. It can consist of decoding and dequantization to restore the input signals' scale.

2nd Stage Processing

Further processing to a high-level representation for evaluation of the different compression algorithms. This step consists of several substeps, which are all performed in double precision:

1. **Doppler FFT with windowing over the Doppler dimension** - To extract the velocity information about the targets, the Doppler FFT is performed on the windowed range FFT data, as described in Subsection 2.2.1. The output is divided by the number of ramps.
2. **Non-coherent integration (NCI) of all RX channels** - This step improves the SNR.
3. **Peak detection** - To detect the peaks, a combination of local maxima detection with global thresholding based on noise estimation is used.
4. **Detection list** - As a result, we get a list of all detected objects.

Comparator

The comparator component compares the reference output with the results of the compression algorithms after decompression. An explanation of which metrics are compared is provided in Subsection 3.4.2.

Results

The results are a printed list of the compared values. The values that are printed are described in Subsection 3.4.2.

3.2.2 Input Data

The input data we used for the simulation is a data cube of raw ADC data. It is representative data captured using a real radar sensor or synthetically generated. The data width of the raw ADC data is 10, 12, or 14 bits.

Real-World Data

The real-world data was collected using the CTRX in different scenarios. Such a scenario would be a simple one with just one stationary target or a more complex one with multiple moving targets in the scene. We use all the different data sets for simulation and evaluation to gain confidence that the compression algorithm can handle various scenarios. Table 3.1 shows the configuration for all measurements.

Parameter	Value
Number of MMICs	1
Samples	512
Ramps	512
RX channels	4
IF data bit width	14 bits

Table 3.1: Configuration parameters for the real-world measurements

In this work, we take real-world measurements to show the compression effects. We use the data with the $id = 1$ for showcasing, which we see in the following Figures. In Figure 3.2,

we can see the range FFT output over ramp 1 and RX antenna 1. Figure 3.4 shows us three heatmaps at different processing pipeline stages. We can see that each processing step lowers the noise and thus increases the SNR. In Figure 3.3 we can see again the range-Doppler map as a 3D image. It is the same data as in Figure 3.4 (c).

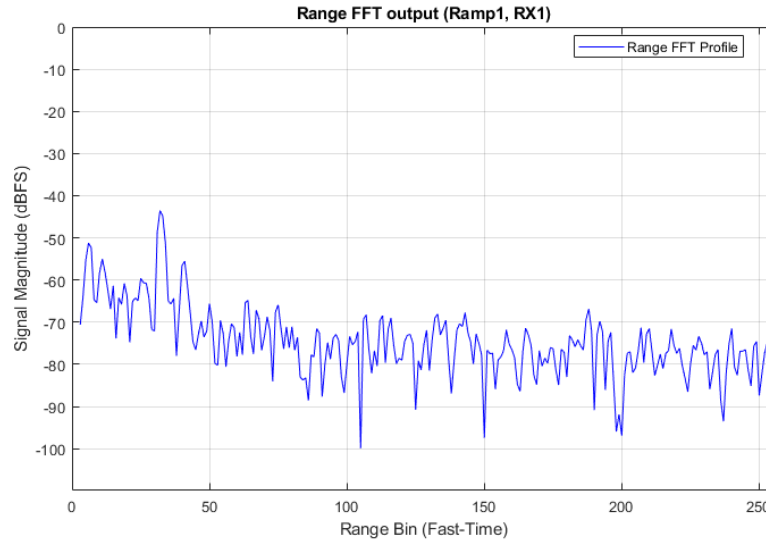


Figure 3.2: Real-world data: Range FFT output for ramp 1 and RX antenna 1

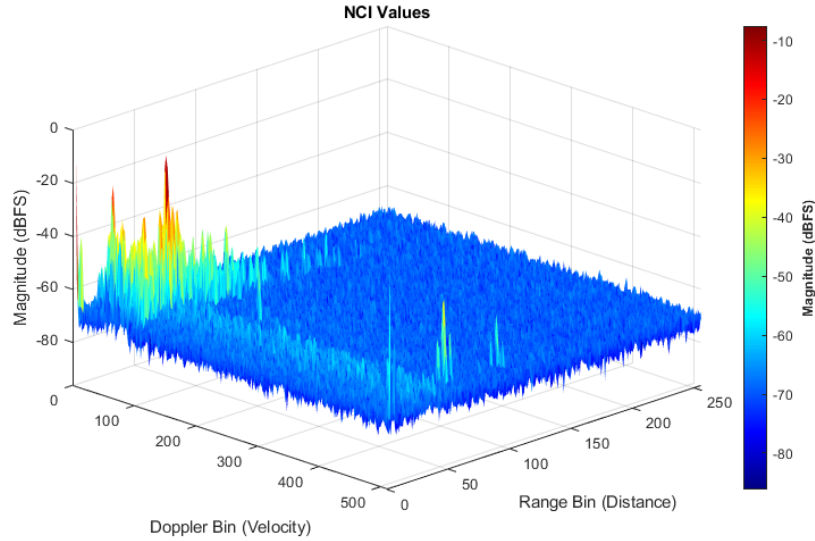


Figure 3.3: Real-world data: 3D range-Doppler image after NCI

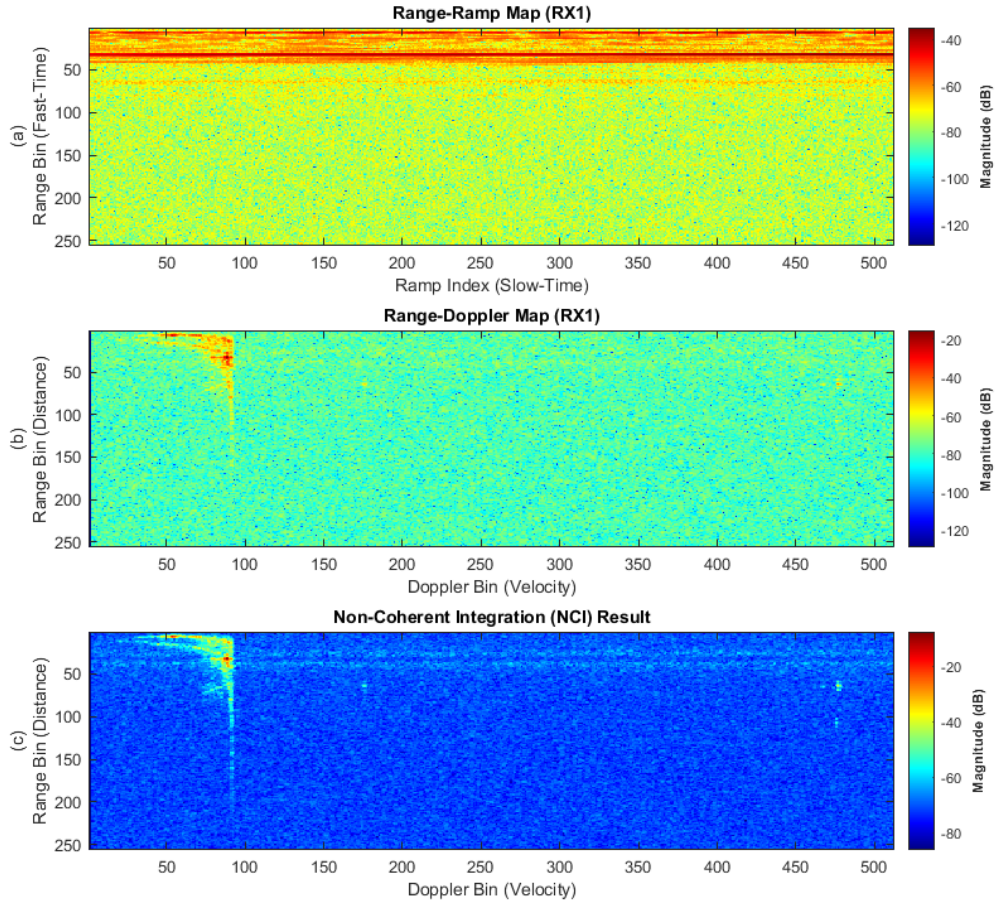


Figure 3.4: Real-world data: 2D heatmaps at the different steps in the processing pipeline. (a) Range-ramp map after the range FFT computation (b) Range-Doppler map after the Doppler FFT computation (c) Range-Doppler map after performing NCI

Simulated Data

We use synthetic data to simulate scenarios and configurations where we don't have real-world data available. This data is generated based on specified configurations. It is possible to specify:

- Number of targets
- Target frequencies for each target
- Target speeds for each target
- Target amplitudes for each target
- Noise level in dB
- Bit width of the raw data
- Number of RX channels
- Number of ramps
- Number of samples

The modeled noise in the data is additive white Gaussian noise (AWGN), and the bit width refers to either 10, 12, or 14 for the bit width of the ADC values. In the Figures 3.5, 3.7, 3.6 we can see the outputs along the processing pipeline. The simulated scenario was configured with the values in the Table 3.2. Using this configuration, we have a complex scenario with two targets at the same distance with different speeds and two targets with the same speed at different distances.

Parameter	Value
Number of targets	5
Target frequencies (range bin)	[100, 150, 150, 200, 400]
Target speeds (doppler bin)	[50, 50, 100, 200, 250]
Target amplitudes	[0.8, 0.3, 0.4, 0.5, 0.7]
Noise level (after NCI)	-75 dBFS
Number of MMICs	1
Samples	1024
Ramps	512
RX channels	4
IF data bit width	14 bits

Table 3.2: Configuration parameters for the simulated data generation

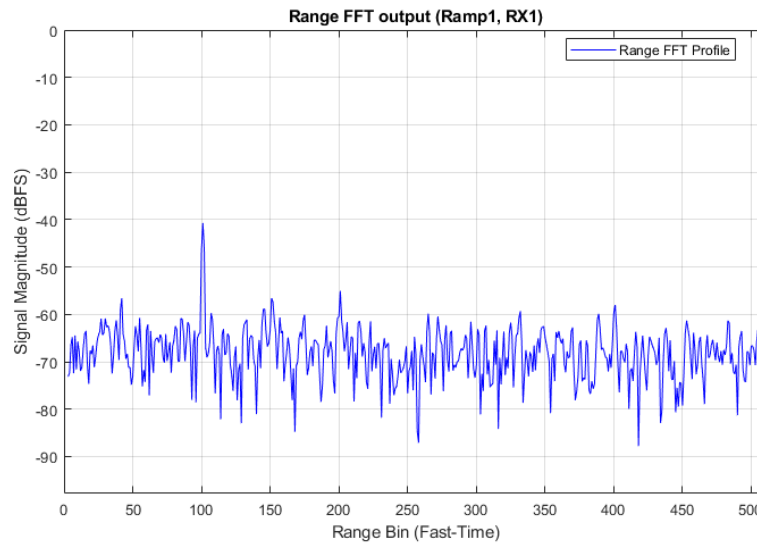


Figure 3.5: Simulated data: Range FFT output for ramp 1 and RX antenna 1

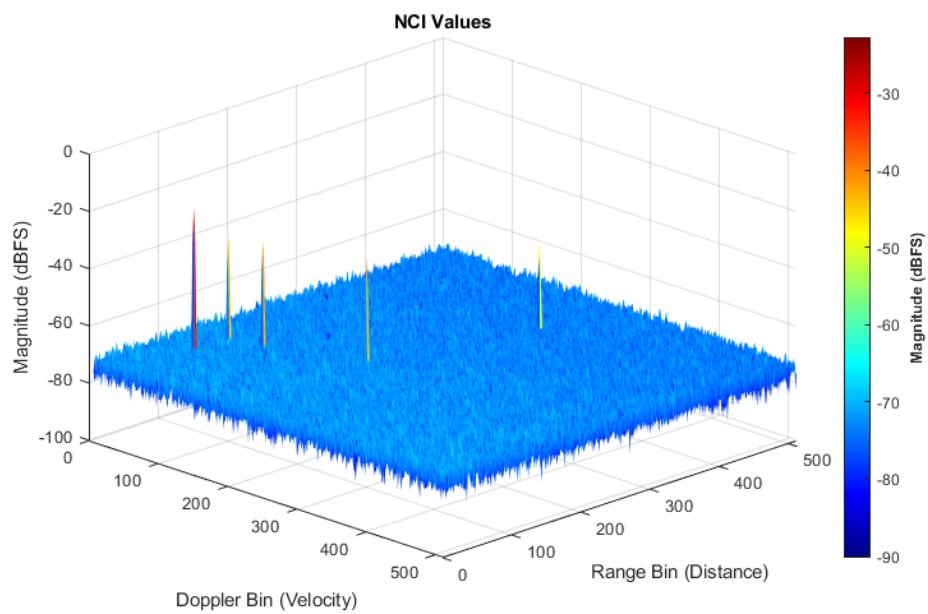


Figure 3.6: Simulated data: 3D range-Doppler image after NCI

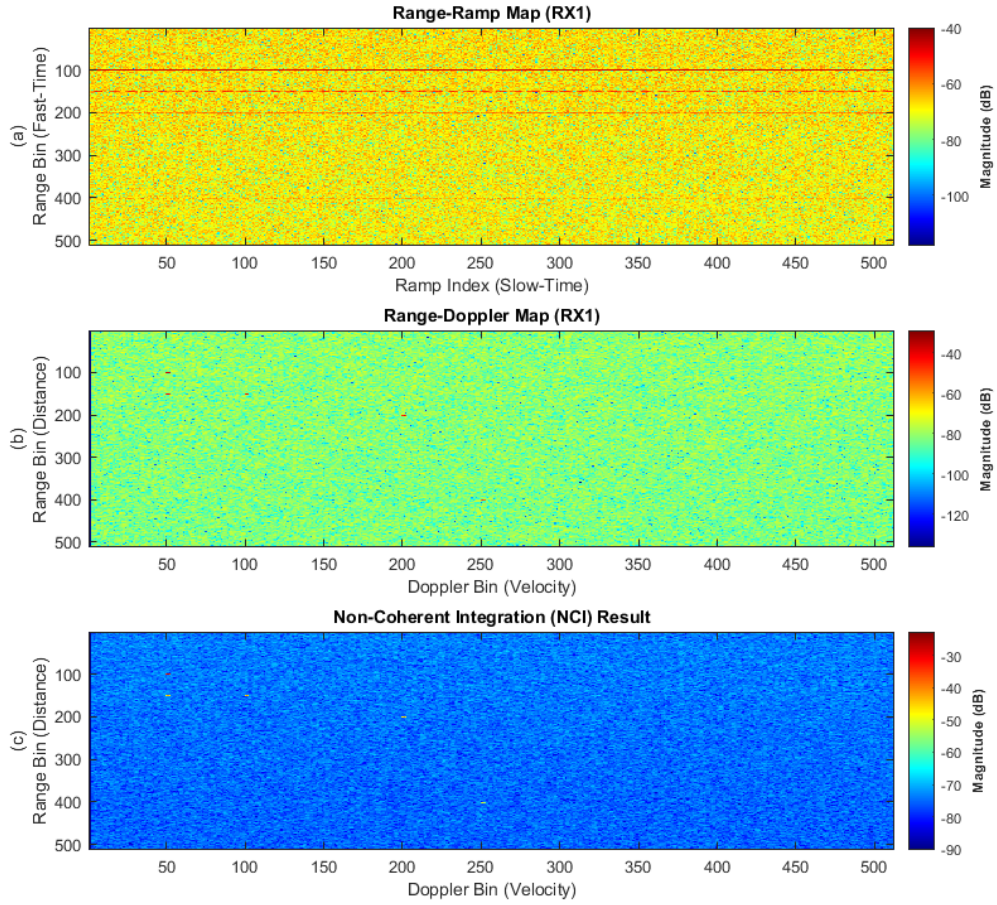


Figure 3.7: Simulated data: 2D heatmaps at the different steps in the processing pipeline. (a) Range-Ramp map after the range FFT computation (b) Range-Doppler map after the Doppler FFT computation (c) Range-Doppler map after performing NCI

3.3 Use Case and Requirements

To be able to define the requirements for the performance of the compression system, we first need to define a realistic use case. The work's primary focus is on compressing the data for transmission to reduce the bandwidth, mainly for future systems integrated in a centralized and zonal architecture. We used the realistic example ramp scenario parameters listed in Table 3.3 to define the use case.

Parameter	Value
Number of MMICs	1
Samples	1024
Ramps	1024
RX channels	4
IF data bit width	12 bits
Output sample rate (OSR)	50 MS/s

Table 3.3: Relevant example configuration Parameters for the ramp scenario

Using these parameters, the resulting data cube size can be calculated as follows:

$$\begin{aligned}
 \text{Data Cube Size (bits)} &= \text{ramps} \times \text{samples} \times \text{IF data bit width} \times \text{RX channels} \\
 &= 1024 \times 1024 \times 12 \times 4 = 50\,331\,648 \text{ bits} \\
 \text{Data Cube Size (bytes)} &= \frac{50\,331\,648}{8} = 6 \text{ MB}
 \end{aligned} \tag{3.1}$$

Since the primary goal is not to develop a compression algorithm to reduce needed storage resources but to reduce transmission bandwidth, the resulting bitrate of this use case is more interesting.

The resulting bitrate can be calculated as follows:

$$\begin{aligned}
 \text{Bitrate (Single RX channel)} &= \text{OSR} \times \text{IF data bit width} \\
 &= 50 \text{ MS/s} \times 12 \text{ bit} = 600 \text{ Mbit/s per RX channel}
 \end{aligned} \tag{3.2}$$

$$\text{Bitrate (All four RX channels)} = 600 \text{ Mbit/s} \times 4 = 2400 \text{ Mbit/s} = 2.4 \text{ Gbit/s}$$

The resulting compression method aims to achieve a compression ratio (CR) to transmit the bitrate of the use case over Gigabit Ethernet via 1000BASE-T1. 1000BASE-T1 is standardized in IEEE Standard 802.3bp-2016 [15] and can transmit data with a speed of 1000 Mbit/s over a single twisted pair cable. IEEE has already published an automotive-suitable physical layer specification for 10 Mbit/s, 2.5 Gbit/s, 5 Gbit/s, and 10 Gbit/s [21] under IEEE 802.3ch. However, the ecosystem is still in the early adoption stage compared to the more established 100 Mbit/s or 1 Gbit/s Ethernet technologies, which are already mature. The faster technologies are more expensive and more challenging to implement. So, given these facts, we decided to work with the Gigabit Ethernet technology. The required compression ratio to transmit the resulting bitrate of 2.4 Gbit/s over the Gigabit Ethernet cable is a CR of 2.4.

Regarding the speed performance requirements, the sensor operates at a cycle frequency of 20 Hz, corresponding to a cycle time of 50 ms. Let's take the above scenario

configuration to calculate the time to acquire one frame.

$$\begin{aligned} \text{Acquisition time} &= \frac{\text{samples} \times \text{ramps}}{\text{OSR}} \\ &= \frac{1024 \times 1024}{50} = 0.02097152 \text{ seconds} \approx 21 \text{ ms} \end{aligned} \quad (3.3)$$

As shown in 3.3, the sensor takes around 21 ms to acquire one frame. After that, there are 29 ms available to perform the digital signal processing (DSP) work, where typically the MCU processes the data. To leave enough buffer time for downstream tasks or system uncertainties, we define that the compression needs to be completed within 15 ms.

3.4 Design Space Exploration

This section describes the design space exploration with the final choice of the most suitable algorithm for implementation on an FPGA. The exploration first defines the performance metrics and the design space. Then, the exploration method is introduced, and the trade-offs between the performance metrics are evaluated, leading to the final design decision.

3.4.1 Design Space

To explore, we first need to define the design space. In Section 2.7, we introduced the state-of-the-art automotive radar data compression solutions, which we will use to determine the design space for this work. We take all these solutions except the LPC-Huffman compression algorithm into the design space. LPC-Huffman compression is not taken because it has the same approach as DRHE, but instead of compressing range FFT data, it operates on raw ADC data; the results for DRHE are more promising.

Algorithm	Method	On-the-fly	Lossless
FP16		✓	✓
CMPRA	Common exponent compression (16 complex bins packed into 256-bit word)	~	×
CMPRB	Common exponent compression (32 complex bins packed into 256-bit word)	~	×
EGE10	Exponential Golomb coding (order 10)	~	×
EGE8	Exponential Golomb coding (order 8)	~	×
EGE6	Exponential Golomb coding (order 6)	~	×
EGE4	Exponential Golomb coding (order 4)	~	×
DRHE	LPC on range FFT data, encode using RLE and Huffman coding	✓	×

Table 3.4: State-of-the-art solutions used to define the design space

In Table 3.4, the design space is defined with a short description of the method. The used symbols have the following meanings: × (no), ~ (partially), ✓ (yes). As a reference data type, we take a 16-bit floating point. CMPRA, CMPRB, and the EGE variants must buffer the data over several ramps. Therefore, they are defined as partially on-the-fly. DRHE can be performed on-the-fly without any buffering. As shown in Table 3.4, none of the schemes is guaranteed to be lossless. CMPRA and CMPRB have a fixed CR and are potentially lossless, but some bits need to be dropped in some cases. The EGE variants are configured to a fixed CR; if the CR is not met, some least significant bits (LSB) are dropped. DRHE is based on Huffman coding, and to make this solution usable, the data

needs to be quantized to 16-bit fixed-point values, which means loss of information. After the quantization process, the scheme is lossless.

3.4.2 Evaluation Metrics

We select the following five evaluation metrics to evaluate the performance of each compression algorithm in the MATLAB simulation environment, and finally select one approach based on these metrics.

Compression ratio (CR) measures the efficiency of the compression algorithm in reducing the data in size. Preferably, the CR should be high without losing critical information about the data.

False negative detection (FN) evaluates the number of real targets missed by compression artifacts and data loss. This value should be minimized to achieve a high data quality.

False positive detection (FP) evaluates the number of incorrectly detected targets. Compression artifacts and loss of information cause it. This metric must be minimized to achieve high data quality.

Noise floor estimation (NF est.) measures the impact of the compression algorithm on the noise floor. The noise floor is calculated by taking the mean of the noisy part of the signal in dB. The results are, therefore, expressed in dB. The impact of the compression algorithm on the noise floor should be minimized to keep the quality of the signal good.

SNR measures the impact of the compression algorithm on the ratio between signal quality and background noise. It is calculated using the RMS signal in dB and the RMS noise in dB. The signal is taken from the detected peaks. The SNR is calculated by $\text{snr} = \text{signalRMSdB} - \text{noiseRMSdB}$ where the variable `noiseRMSdB` equals the noise floor. The result is expressed in dB. A compression algorithm should maintain a high SNR for good signal quality.

3.4.3 Exploration Process

The exploration process involves multiple steps. In Figure 3.8, the process is shown as a block diagram.

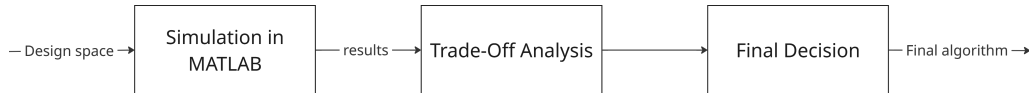


Figure 3.8: Exploration process block diagram

The first step is to simulate all the different solutions within the design space using the MATLAB environment, described in Section 3.2. We obtain the results from the simulation and compare them to the reference results. The second step is to perform a trade-off analysis to identify trade-offs, such as a higher compression ratio leading to lower data fidelity. The last step is to select the solution we will continue working with.

3.4.4 Exploration Results

In the design space exploration, we simulated the different radar data compression solutions within the design space, which we defined in Subsection 3.4.1. As already introduced in Section 3.2, a MATLAB environment is used for exploration. In Table 3.5, the performance of the various algorithms using the metrics defined in Subsection 3.4.2 can be seen.

Algorithm	CR	FN (%)	FP (%)	NF est. (dBFS)	SNR (dB)
FP16	1	0	0	-70.714	22.407
FX16	1	0.59	1.18	-70.691	22.33
CMPRA	2	0.59	0.59	-70.687	22.418
CMPRB	4	7.69	4.73	-70.253	22.111
EGE10	1.6	1.18	1.18	-70.689	22.403
EGE8	2	2.37	2.37	-70.64	22.289
EGE6	2.67	7.69	7.1	-70.367	22.197
EGE4	4	40.24	26.63	-74.689	28.245
DRHE	3.25	0.59	1.18	-70.691	22.33

Table 3.5: Design space exploration results using the MATLAB simulation environment

As reference data, we take the format **FP16**, and in the row with the algorithm **FX16**, we can see the performance if we decide on a fixed-point hardware implementation. Additionally, some other algorithms need quantization of the data type *int16* to compare these to the values of **FX16**. The algorithm **CMPRA** maintains good data fidelity and minimally raises the noise floor and lowers the SNR, with a decent CR of 2, which still does not fulfill our requirement introduced in Section 3.3. **CMPRB** provides twice the amount of CR, but the target detection performance is less promising. This algorithm should be used for specific applications where high data fidelity is not crucial. The **EGE**-variants **EGE10** and **EGE8** achieve less or equal CR than **CMPRA** with worse detection performance, and the variants **EGE6** and **EGE4** achieve a higher CR than the requirement of $CR = 2.4$ for this work, but the amount of information loss is high, as we can see in the resulting metrics. The algorithm **DRHE** is the only algorithm that achieves a CR higher than the requirement with high data fidelity. For **DRHE**, the data needs to be quantized to **FX16**, and after that step, the algorithm is lossless. Even if some major adaptations need to be done to make the algorithm usable for practice, such as guaranteeing a fixed CR, based on the results in Table 3.5, we decide to continue this work with the **DRHE** algorithm.

3.4.5 Final Design Choices

Based on the design space exploration, we select the **DRHE** algorithm as the most suitable compression technique for this work. It achieves a high CR of 3.25 while preserving critical radar information with false negative detections of 0.59 % and false positive detections of 1.18 %. The noise floor is raised by a small amount compared to the **FP16** format, from -70.714 dBFS to -70.691 dBFS, and the SNR is also reduced by a small amount from 22.407 dB to 22.33 dB.

3.5 Final Algorithm Design and Optimization

The final compression algorithm was refined and optimized to address limitations identified during the design space exploration and prepare it for hardware implementation. Two key modifications were made. The RLE part of the final algorithm was removed in Subsection 3.5.1, and the Huffman dictionary was fixed in Subsection 3.5.2.

3.5.1 Remove Redundancy

As introduced in Subsection 2.7.3, the DRHE algorithm uses a hybrid combination of RLE and Huffman coding. In Figure 2.10, the encoding process is shown. The input data is 16-bit difference data, and each value in the input data stream will be encoded into a binary form of [RRRRSSSS+APPEND]. We call the RRRR part R4 for convenience and the SSSS part S4. The APPEND field is an SSSS-bit field that contains the actual difference value. The R4 value consists of the number of zero values preceding a non-zero value. S4 is the region in which the non-zero data is located. The table of regions is shown in Figure 2.10. The 8-bit R4S4 value is encoded using Huffman coding.

During the design space exploration, it was found that the RLE part contributes minimal improvement, if any, to the compression performance while adding unnecessary complexity. Especially in real-time processing requirements, every complexity that can be discarded should be discarded. In the paper [13], the authors stated that the main characteristic of the difference data is that there are many zeros in the data, and the second main characteristic is that the small values are much more frequent than the large values. Using the real-world radar data, we obtained the same characteristics. The histogram of the difference values is shown in Figures 3.9 and 3.10. The difference values show in both the real and imaginary part a normal distribution shape. The histograms show the value range from -40 to 40 because most values are in that range.

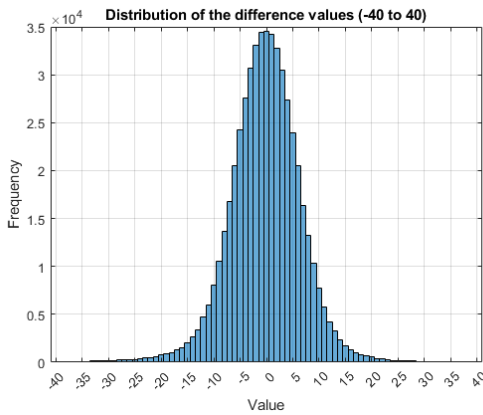


Figure 3.9: Difference values histogram for the real part

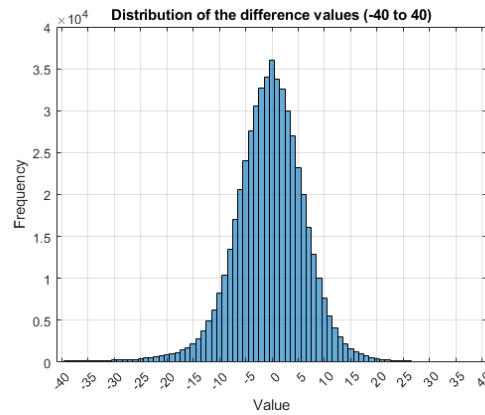


Figure 3.10: Difference values histogram for the imaginary part

Figure 3.12 shows the histogram for the resulting R4S4 symbol values for the real part. We can see that the distribution of the values is far from the optimal case, as shown in Figure 3.11.

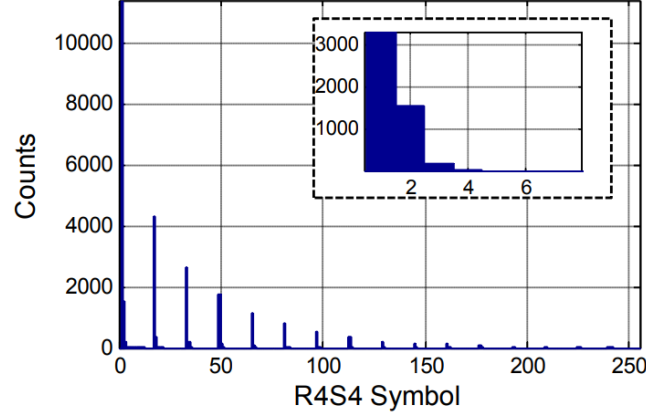


Figure 3.11: Optimal R4S4 symbol histogram from the reference paper [12]

Since we need to use a fixed Huffman dictionary to make a hardware implementation feasible, it is impossible to fix it with values that are far from the optimal case and still achieve good performance. To tackle this problem, a possible solution is to decrease the number of symbols in the Huffman dictionary. Since the S4 bits are the most essential part of the algorithm, we decided to simulate the algorithm without the RLE part. So, considering just the SSSS part, each zero value encodes to SSSS = 0. In Figure 3.13, the distribution of the S4 values is shown as a histogram, and in Table 3.6, the results for the simulation with the same input data are shown. So, comparing the results in Tables 3.5 and 3.6, we can see that the performance results are the same except for the CR, which decreased from 3.25 to 3.24, which is an acceptable effect for the improvements. With this optimization, the size of the fixed Huffman dictionary, which has to be stored on the hardware, can be reduced from 2^8 to 2^4 values. Additionally, it improves the worst-case performance since the longest Huffman code for a dictionary with 2^8 entries is much longer than the one for a dictionary with 2^4 entries. If the difference data's distribution does not match the fixed Huffman dictionary, the performance is better if using just the S4 part.

Algorithm	CR	FN (%)	FP (%)	NF est. (dBFS)	SNR (dB)
FP16	1	0	0	-70.714	22.407
FX16	1	0.59	1.18	-70.691	22.33
DRHE	3.25	0.59	1.18	-70.691	22.33
DRHE (without RLE)	3.24	0.59	1.18	-70.691	22.33

Table 3.6: Simulation results for DRHE without RLE

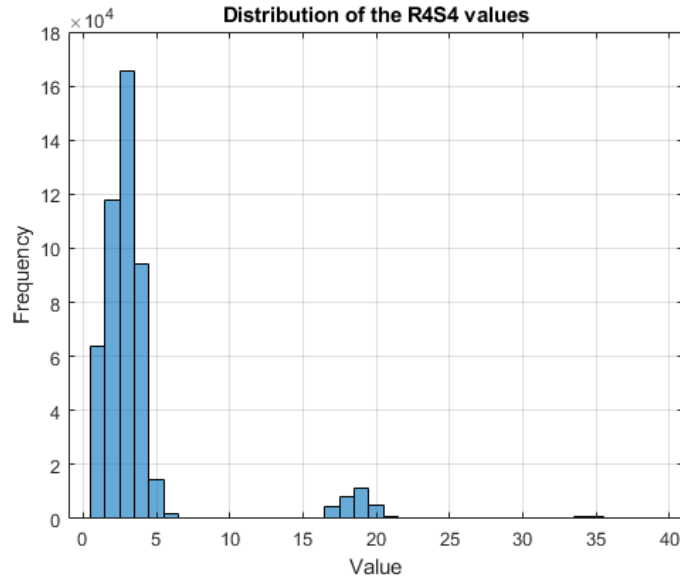


Figure 3.12: R4S4 symbol histogram for the real part (0 to 40)

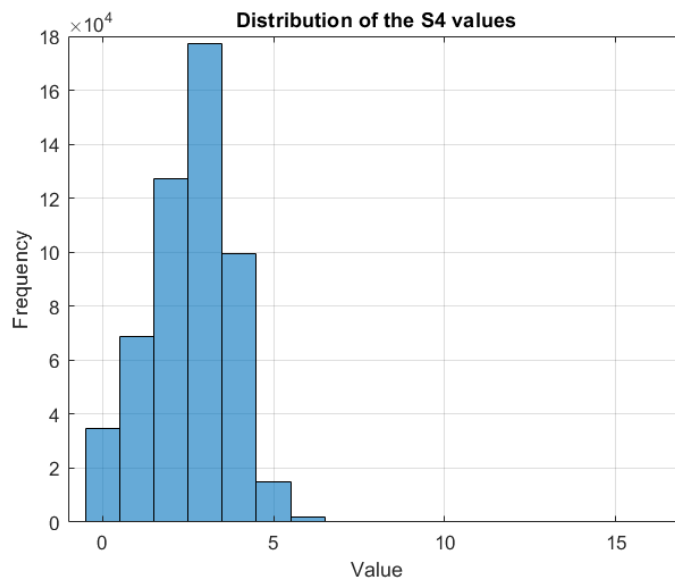


Figure 3.13: S4 symbol histogram for the real part

3.5.2 Fixed Huffman Dictionary

To adapt the DRHE algorithm for hardware implementation, the Huffman dictionary is fixed to a pre-computed configuration. This change significantly reduces the algorithm's complexity by eliminating the need for dynamic dictionary generation for every input, which is resource-intensive and unsuitable for real-time, embedded systems [27]. Additionally, it eliminates the need to transmit the Huffman dictionary, which needs additional bandwidth and thus reduces the CR significantly.

Fixing the Huffman dictionary can lead to a decrease in the CR, because the fixed dictionary may not represent the value distribution of the input perfectly. Still, we get a lot of complexity elimination in exchange. The algorithm's performance using the fixed Huffman dictionary was verified by simulating the available test data. For this work, we used the generated dictionary for the real part of the S4 values for the real radar data with $id = 1$, as described in Subsection 3.2.2, which we introduced in Subsection 3.2.2. The histogram for the S4 symbols is shown in Figure 3.14, and the fixed Huffman dictionary that is used for simulation and in the hardware implementation is listed in Appendix A in Table A.

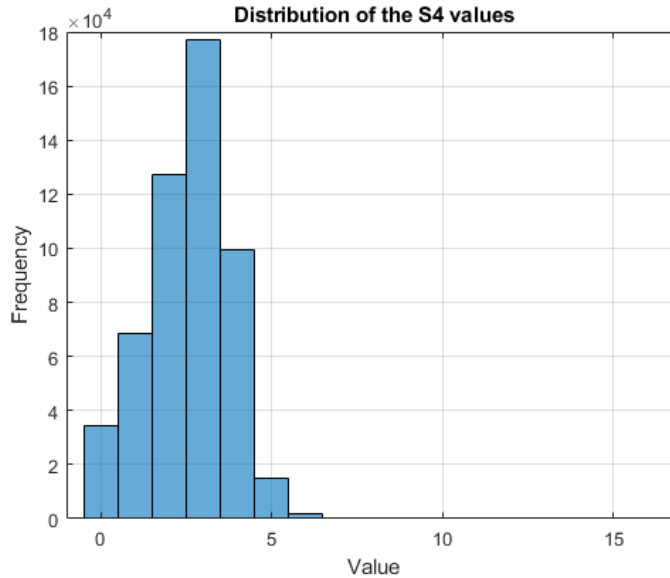


Figure 3.14: Fixed Huffman dictionary used in this work. The used input data is $id = 1$.

3.5.3 Final Compression Scheme

In Subsection 2.7.3, the DRHE algorithm is introduced in its original version. In the previous Subsections 3.5.1 and 3.5.2, the algorithm was refined and optimized to match the needs for this work. In this section, the final algorithm, which is implemented on an FPGA, is described in more detail.

The DRHE algorithm exploits the sparsity of the data in the Doppler dimension. Figure 3.15 shows the basic elements of the compression scheme. The preprocessed ADC data is passed through an FFT, and the resulting data is streamed ramp after ramp into

the compression core. The compression core can be divided into smaller blocks: Model prediction and encoding.

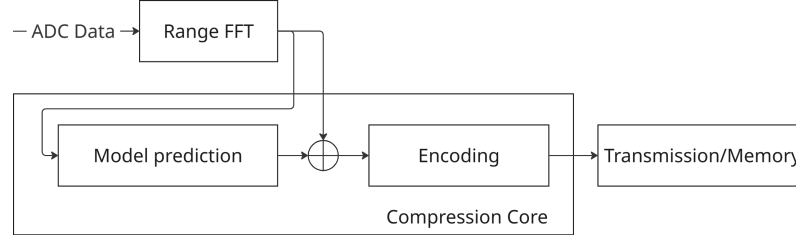


Figure 3.15: Top-level block diagram of the DRHE compression scheme

Model Prediction

As introduced in the state-of-the-art section, the model prediction is performed using two IIR filters for the magnitude and the phase. The formula for the prediction is:

$$\hat{X}[m, n] = \hat{a}[m, n]e^{j\hat{\theta}} \quad (3.4)$$

where $\hat{a}$ and $\hat{\theta}$ are the predicted magnitude and phase, respectively. The magnitude prediction is defined as:

$$\hat{a}[m, n] = \alpha \hat{a}[m-1, n] + (1 - \alpha)a[m-1, n] \quad (3.5)$$

The phase prediction is defined as:

$$\beta \hat{\theta}[m-1, n] + (2 - \beta)\theta[m-1, n] - \theta[m-2, n] \quad (3.6)$$

The parameters α and β are tunable parameters that control how fast the predictions are influenced and modified by the real input signal. In Figure 3.16, the “unfolded” model prediction block is shown in more detail, including the conversion from Cartesian form, which we get as output from the range FFT, to polar form and vice-versa. The data

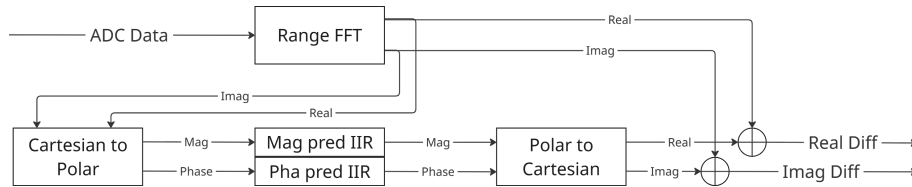


Figure 3.16: Block diagram of the DRHE model prediction block

flows from the range FFT into the conversion from Cartesian to polar form. The model prediction is performed using the previously described IIR filter, using the magnitude and phase, respectively. The predicted values are then converted back to the Cartesian form, and as the last step, the difference between the predicted and real values is calculated using

$$X[m, n] - \hat{X}[m, n] \quad (3.7)$$

The resulting difference is in the 16-bit integer format and is further encoded in the next step.

Encoding

As discussed in the above sections, we modified the encoding algorithm by removing the RRRR bits and encoding just the SSSS bits. The SSSS field is the 4-bit field that specifies the region where the difference value is located. The APPEND field is a SSSS-bit field that contains the actual difference value. Table 3.7 shows the mapping from the difference values to the S4 values. To pack the difference value into S4 number of bits, the input data is shifted. To negative values, $2^{\text{SSSS}} - 1$ is added. For example, the difference value -5 is located in the region 3, so $\text{SSSS} = 0011$ and APPEND is calculated using $-5 + 2^3 - 1 = 2$, so $\text{APPEND} = 010$. As a result, the shifted negative values result in APPEND bits starting with 0 as the first bit, and the positive numbers start with a 1. In Figure 3.17, the shifting process to generate the APPEND bits is shown. The S4 field is encoded using

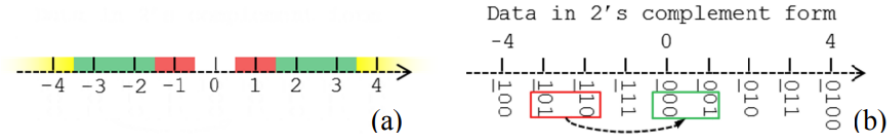


Figure 3.17: (a) Ranges (b) Data shifting for the APPEND bits, the underscores denote the sign bits [12]

Huffman coding as a final encoding step. As discussed in Subsection 3.5.2, the Huffman dictionary is pre-computed and stored in memory in the final implementation. So basically, the encoding process is simply performing a look-up in the fixed Huffman dictionary.

SSSS	Value range	Range size
0	0	1
1	-1, 1	2
2	-3, -2, 2, 3	4
3	$[-7, -4], [4, 7]$	8
4	$[-15, -8], [8, 15]$	16
5	$[-31, -16], [16, 31]$	32
...	$[-2^n + 1, 2^{n-1}], [2^{n-1}, 2^n - 1]$	2^{SSSS}
15	$[-32767, -16384], [16384, 32767]$	32768

Table 3.7: Mapping from value range to the SSSS value

3.5.4 Decompression Scheme

As we described above, we also need to define the decompression scheme for the final compression scheme. In this work, we decompress the data using the MATLAB simulation environment. The decompression process is defined by inverse compression operations in reverse order. In Figure 3.18, the process can be seen as a top-level view block diagram. The compressed difference data is the input to the decompression core, which can be divided into two main blocks: Decoding and inverse model prediction.

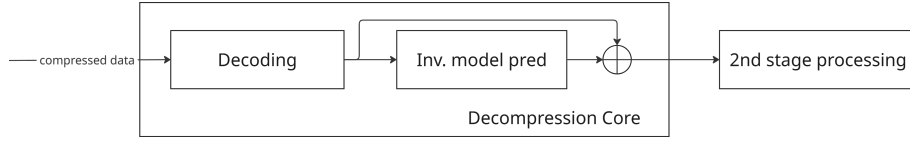


Figure 3.18: Basic decompression scheme block diagram

Decoding

The decoding is performed by reading the input one bit after the other and checking the Huffman dictionary for a match each time. This works because no code is a prefix of another code. If a match exists, we can decode the Huffman code to the SSSS (S4) value. Reading the next S4 number of bits from the data stream is the second step in deriving the APPEND field. The original difference value is negative if the APPEND bits start with a 0-bit. So we must subtract $2^{\text{SSSS}} - 1$ from the APPEND-bits. As an example, assume $\text{SSSS} = 3$ and $\text{APPEND} = 010$. The APPEND-bits are starting with a 0, so we need to calculate $\text{APPEND} - (2^3 - 1) = 2 - (2^3 - 1) = -5$ to get the decoded difference value. For positive numbers, the APPEND-bits start with a 1 and thus can be taken directly as they are. The decoded difference value is processed further by the inverse model prediction block.

Inverse Model Prediction

The inverse model prediction maps the decoded difference values back to the range FFT data. In Figure 3.19, the process is shown as a block diagram. The input is the difference data, which is added to the prediction output from the model. The model is initialized with zero, so for the first ramp, the difference is added to 0. After the addition, the restored range-FFT data is the result and the input for the prediction model. The same formulas define the model as in the compression scheme in Subsection 3.5.3.

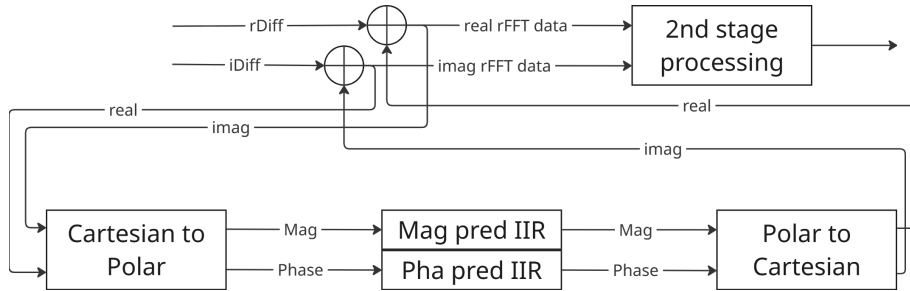


Figure 3.19: Inverse model prediction for the decompression process

The resulting decompressed range FFT data is restored without any loss as 16-bit integer values. It can be directly passed through the 2nd stage processing.

Chapter 4

Implementation

This section describes the hardware implementation of the final algorithm chosen in Section 3.4 and optimized in Section 3.5, called DRHE. The implementation aims to achieve a real-time compression implementation while keeping the used hardware resources in a reasonable range. As introduced in Section 2.4, the FPGA technology was chosen due to its reconfigurability, parallel processing capabilities, and suitability for real-time radar data processing in automotive systems. This section introduces the hardware setup, the development flow, and finally, the hardware implementation design with its challenges.

4.1 Hardware Setup

The algorithm is implemented on the AMD Zynq™ UltraScale+™ MPSoC platform, which is the heart of the setup. The MPSoC is mounted on the Trenz TE0820-04 System-on-Module (SoM), and this is mounted on a board that we call the interface board. The interface board is used to provide several hardware interfaces to connect to. In the following section, some more details about each component are provided. In Figure 4.1, the interface board is shown.

AMD Zynq™ UltraScale+™ MPSoC

The exact used part is the XCZU3CG-SFVC784-1-e, which combines PL, which is the FPGA, with a feature-rich 64-bit dual-core advanced RISC machines (ARM) Cortex-A53 and dual-core ARM Cortex-R5F based PS [34]. The PL provides 154,350 system logic cells, CLBs consisting of 141,120 flip-flops (FF) and 70,560 look-up tables (LUT). Furthermore, it provides 360 DSP Slices, 7.6 Mb of on-chip block RAM, and 1.8 Mb of on-chip distributed RAM. The listed features are enough to implement the compression pipeline.

Trenz TE0820-04 SoM

The TE0820 is a SoM integrating the AMD Zynq™ UltraScale+™ MPSoC with 2 GB DDR4 SDRAM, 2 × 64 MB Serial Flash, 8 GB eMMC, as well as powerful switching power supplies for all required voltages. The module is equipped with a Lattice Mach XO2 CPLD for system control. Three high-speed connectors provide a large number of inputs

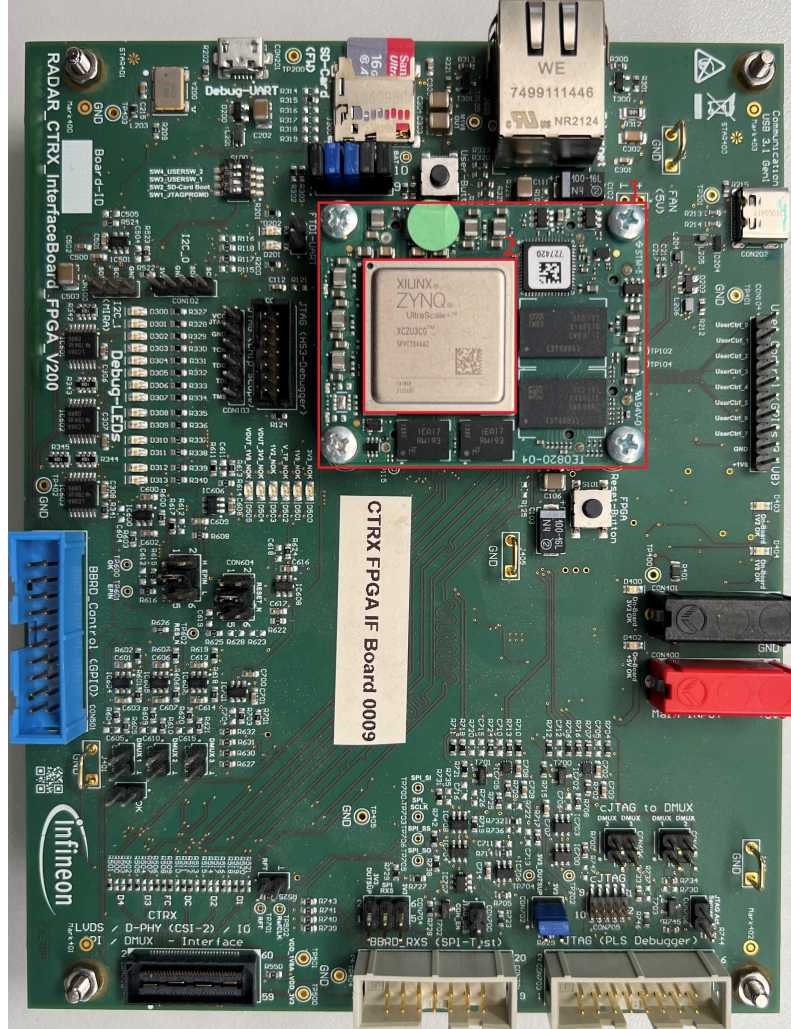


Figure 4.1: Interface board used for the implementation. (1) Trenz TE0820-04 SoM (2) AMD Zynq™ UltraScale+™ MPSoC

and outputs. Additionally, the module offers Gigabit Ethernet and USB 2.0 transceivers. The module size is smaller than a credit card and can be exchanged easily with other compatible modules [36].

Interface Board

The interface board, which is designed by IFX, provides several necessary hardware connectors and pins for this work. For this work, we mostly used the JTAG interface for debugging and programming of the FPGA, the serial debug port for logging and debugging, the SD card module for storing/loading the raw ADC data, and the GPIO pins for testing and validating. Furthermore, the interface board enables the connection between the CTRX radar MMIC and the FPGA platform through an FPGA mezzanine card (FMC) connector.

4.2 Development Tools

To implement the radar data compression system, the AMD Vitis software platform 2022.2, including Vitis[™] Embedded and Vitis high-level synthesis (HLS), and the AMD Vivado Design Suite 2022.2 are used. These tools work seamlessly together, so integrating the software and hardware on the FPGA is no problem.

Vitis Embedded is a standalone embedded software development package for developing the bare-metal host application running on the embedded ARM processors. It provides a platform to write and debug software directly for the ARM Cortex processors in the Zynq[™] UltraScale+[™] MPSoC.

Vitis HLS allows for the easy creation of complex FPGA algorithms by synthesizing a C/C++ function into register transfer level (RTL). It is used because the model prediction includes some very complex mathematical functions that are available in the provided libraries and can be synthesized easily. We used it by manually translating the MATLAB implementation of the final compression algorithm into C/C++ functions and deriving the hardware implementation through high-level synthesis. The tool is tightly integrated with Vivado Design Suite and the Vitis unified software platform, allowing rapid prototyping and optimization of the compression IP core. Vitis HLS reports timing, performance, and resource usage estimates. This enables optimization without leaving Vitis HLS.

Vivado design suite integrates and connects the different IP cores into the FPGA design. It is responsible for the FPGA design synthesis, implementation, and bitstream generation. The tool reports the final resource usage, timing values, and all other needed values for evaluation.

The development flow was an iterative process, with refinements based on the testing results.

4.2.1 Simulation, Testing, and Verification

This section describes the simulation, testing, and verification at different stages of the development process. Different tools and techniques are used to ensure functionality, correctness, and real-time performance.

Simulation in Vitis HLS was used to verify the functionality of the compression algorithm at the C++/HLS level. The C simulation ensures that the C/C++ implementation matches the expected behavior. The testbench functionality was compared and verified once manually using the MATLAB simulation output. The two outputs have a deviation of at most 2 in the difference data because no bit-accurate matching MATLAB and C++ implementation in Vitis HLS was possible. The difference in the data is caused by the different behavior of the fixed-point implementations. The C/RTL Cosimulation ensures that the algorithm on RTL level matches the expected behavior in C/C++ as well, and the cosimulation reports the performance in terms of latency and throughput.

Verification using the integrated logic analyzer (ILA) Core is used to validate the functionality of the IP cores and data flow in the FPGA design. It enables the debugging of internal signals and timing issues after integrating the IP cores into the Vivado block design. Every internal signal can be probed to the ILA core, a specific trigger can be set, and the signal values can be captured and debugged using the Wave Viewer. This approach is beneficial because there is no need to probe the signals to some GPIO pin since it is possible to monitor them internally.

System-level testing in Vitis using the bare-metal application to test the complete hardware-software system, including the hardware components and interfaces. It is loaded onto the ARM Cortex processor, configuring the IP cores and ensuring that the system operates as expected in a real-world scenario with radar input data and produces valid compressed output. Debugging the application from the host system is possible as well.

4.3 Hardware Implementation Design

As introduced in Subsection 3.5.3 and visualized in Figure 3.15, the compression core can be divided into two smaller blocks: The model prediction and the encoding block. The main goal in this work is to implement the compression core, but to verify its functionality and prove the compatibility, we implement a whole system. In Figure 4.2, a simplified top-level block diagram of the system is shown as an initial plan. The system includes loading ADC data from the SD card into the memory using a bare metal program. Then the application should trigger the AXI DMA IP-Core to read the data from the memory, stream it over the AXI Stream interface to the FFT core, and the output is directly streamed to the compression core, which outputs again a stream back to the DMA. Finally, the DMA writes the compressed data back to the memory. In the following subsections, each block is described in more detail.

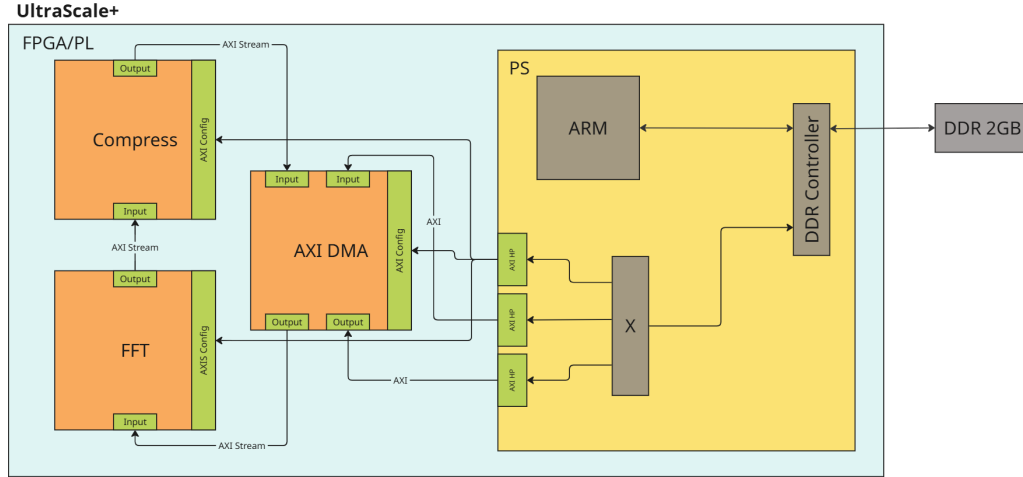


Figure 4.2: Simplified top-level system block diagram

4.3.1 Bare Metal Application

The purpose of the bare metal application is to handle the system's control logic. It loads the ADC data from the SD card and stores it in the memory in the format needed for the FFT IP core. The data is already windowed and shifted to 16-bit full-scale, and the FFT IP core is configured in the 4-channel mode. The application configures the AXI DMA and the Compression IP core over an AXI interface and the FFT IP core over the AXI Stream interface. After the setup and configuration are performed successfully, the AXI DMA is triggered to read $nSamples \times nRamps \times nRXchannels \times sizeof(int16) \times 2$ bytes from the memory where the ADC data is located. After that, the write transfer is also triggered with the same number of bytes. Since the compressed data stream is of variable length, the TLAST signal has to be triggered with the last data on the stream.

4.3.2 AXI DMA IP Core

The AXI DMA IP core v7.1 is available from Xilinx in the Vivado Design Suite. The core is responsible for reading the ADC data from the DDR4 memory, streaming it over the AXI Stream interface to the FFT IP core, receiving the compressed data over the AXI Stream interface, and finally writing the compressed data back into the memory. The AXI DMA must be configured to fit the application's needs. The Scatter Gather Engine is disabled, and the configurations for the read channel and write channel are shown in Tables 4.1 and 4.2.

Name	Value
Number of Channels	1
Memory Map Data Width	128
Stream Data Width	128
Max Burst Size	256

Table 4.1: AXI DMA configuration for the Read Channel

Name	Value
Number of Channels	1
Memory Map Data Width	256
Stream Data Width	256
Max Burst Size	128

Table 4.2: AXI DMA configuration for the Write Channel

4.3.3 FFT IP Core

The FFT IP core v9.1 is available from Xilinx. It computes an N -point forward DFT or inverse DFT where N can be 2^m , $m = 3 - 16$. The essential configuration for the FFT core for this work is listed in Table 4.3. All these configuration parameters can be set dynamically through the AXI Stream interface, but in this work, we used the static configuration listed.

Name	Value
Number of Channels	4
Transform Length	512
Target Clock Frequency (MHz)	100
Architecture Choice	Radix-4, Burst I/O
Data Format	Fixed Point
Scaling Options	Scaled
Rounding Modes	Convergent Rounding
Input Data Width	16
Phase Factor Width	24

Table 4.3: FFT core configuration parameters

We decided to use the fixed-point data format with scaling. To prevent the output from overflowing the data format a scaling schedule needs to be chosen wisely. The scaling schedule is specified with two bits for each stage. The scaling can be specified as 3, 2, 1, or 0, representing the number of bits to be shifted. As an example for $N = 512$, Radix-4 Burst I/O there are $\lceil \log_4(512) \rceil = 5$ stages. We choose to scale by $\log_2(N) = \log_2(512) = 9$ bits, so 9 needs to be distributed over the 5 stages, with the bigger values at the first stages. The resulting scaling schedule is 01 10 10 10 10 for each channel, following 1111 for forward FFT for each channel. The FFT core expects the input data to be in interleaved format over the four channels. In Figure 4.3, the expected format is shown. The data width is 128-bit, and each data transaction contains one complex 16-bit sample for each RX channel. As we have a real-valued signal as input, the imaginary part is set to 0. The output is in the same format as the input; thus, we get a complex 16-bit bin per RX channel per transaction.

```

CHAN_3_XN_IM_0(127:112)
CHAN_3_XN_RE_0(111:96)
CHAN_2_XN_IM_0(95:80)
CHAN_2_XN_RE_0(79:64)
CHAN_1_XN_IM_0(63:48)
CHAN_1_XN_RE_0(47:32)
CHAN_0_XN_IM_0(31:16)
CHAN_0_XN_RE_0(15:0)

```

Figure 4.3: Expected input data format by the FFT IP core

4.3.4 Compression IP Core

As the heart of this work, the compression IP core implements the final algorithm described in Subsection 3.5.3 in hardware. It receives the 16-bit complex input range FFT data over the 128-bit wide AXI Stream interface directly from the FFT core, compresses it on-the-fly, and writes the compressed data to the 256-bit wide AXI Stream. As shown in Figure 3.15 and 3.16, the algorithm can be divided into smaller blocks, which we will use to design the implementation. The implementation in this work is designed based on the use case parameters introduced in Section 3.3 and some common limitations of similar found systems. The number of samples (**nSamples**), the number of ramps (**nRamps**), and the number of RX channels (**nRX**) can be configured over the AXI4-Lite interface. The allowed values, value ranges, and the fixed parameters are listed in Table 4.4.

Name	Value
Number of samples	128, 256, 512, 1024, 2048
Number of ramps	2^n , with $0 \leq n$
Number of RX	1 – 4
Number of TX	1
Alpha (α)	0.6
Beta (β)	0.4

Table 4.4: Configuration parameters and ranges for the FPGA implementation

Based on the largest values for the number of samples, $\text{maxN} = 2048/2 = 1024$, and for the number of RX channels, $\text{maxNRX} = 4$, five static 2D-arrays, which are stored in the BRAM, with the dimensions $\text{maxNRX} \times \text{maxN}$, are created. The number of samples is divided by 2 because we discard the symmetric half of the range FFT output. The arrays store the values for the IIR filter of previous ramps. So there is a separate 1D-array for each RX channel. In the implementation, we used a **counter**, which is taken modulo maxN , for tracking the sample number and indexing the arrays. α and β are fixed values because testing has shown that they have a minimal effect on the algorithm's performance, but for future implementations, they can be made configurable easily. To reduce the resource usage, fixed-point types and arithmetic are used with rounding as the quantization mode, and the default wrap-around mode is used for overflow. The fixed-point data-type declarations are available in the `<ap_fixed.h>` library. The type

`ap_ufixed<16,16,AP_RND>` is unsigned, has 16 bits in total, 16 out of them are for the integer part, with 0 fractional bits, and it is used for storing the magnitude values. We call this type for convenience `mag_data`. For phase values the type `ap_fixed<16,3,AP_RND>`, called `phase_data`, is used, with 16 bits in total, split into 1 sign bit, 2 integer bits, and 13 fractional bits. The range FFT data, both the real and imaginary parts, is stored using the type `ap_fixed<16,16,AP_RND>` called `fft_data`, which corresponds to a signed 16-bit integer.

In Figure 4.4, one iteration of incoming data, model prediction, compressing, and packing into the output buffer is shown, with `nRX = 4`. The input AXI Stream is 128-bit wide and contains, for each transaction, one complex FFT bin for each RX channel. As a first step, the input is split into a separate unsigned 32-bit value for each RX channel. So, at this point, we have independent input data and independent arrays for the model prediction for each of the RX channels. This enables parallelization. In Vitis HLS, it is realized by defining a loop with `maxNRX` iterations and specifying the pragma “unroll”. In Listing 4.1, the used code is shown.

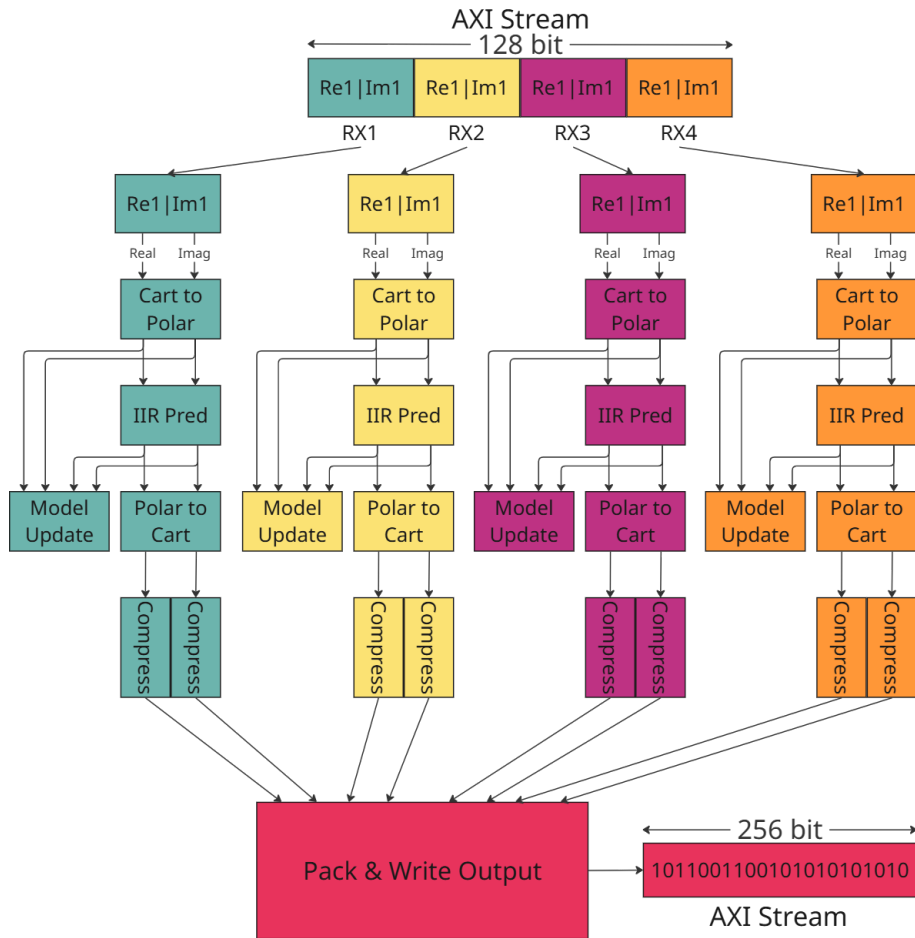


Figure 4.4: Compression pipeline with parallel processing

```

1 pkt_inter rx_s[max_nRX];
2 ap_uint<32> rx_in[max_nRX];
3
4 // Unpack the input stream
5 rx_in[0] = (input_stream.data >> (96)) & 0xFFFFFFFF;
6 rx_in[1] = (input_stream.data >> (64)) & 0xFFFFFFFF;
7 rx_in[2] = (input_stream.data >> (32)) & 0xFFFFFFFF;
8 rx_in[3] = (input_stream.data >> (0)) & 0xFFFFFFFF;
9
10 for (int i = 0; i < max_nRX; i++) {
11 #pragma HLS unroll
12     // Split uint32 to int16 real and imaginary part
13     // Cartesian to Polar conversion
14     // Model prediction
15     // Model update using array at index i
16     // Polar to Cartesian
17     // Calculate difference
18     // Compress into rx_s[i]
19 }
20 // Pack and write to output using rx_s

```

Listing 4.1: Example code: Loop unrolling for parallel processing for each RX channel with relevant stages as comments

The stages that are processed in parallel are similar, as shown in Figure 3.16. Multiple times, mathematical functions from the library `hls_math.h` are needed, which we denote with the prefix “`hls::`”. At first, the 32-bit input value is split into two `fft_data` types for the real and imaginary parts, and these values are converted into polar representation. The magnitude is calculated using the fixed point variant of `hls::sqrt(ap_uint<32>)`. The phase is computed using `hls::atan2(ap_fixed<16,16> y, ap_fixed<16,16> x)`. The next block is the “Model prediction” block. The previous magnitude prediction and the previous magnitude value are loaded by indexing the array at the position `counter`, which is the current sample number. The magnitude prediction is calculated by

```

1 curr_mag_pred = (alpha * prev_mag_pred_val)
2   + ((1 - alpha) * prev_mag_val);

```

and the phase prediction is calculated by

```

1 ap_fixed<32, 4, AP_RND> tmp_first_part =
2   (beta * prev_phase_pred_val);
3 ap_fixed<32, 4, AP_RND> tmp_sec_part =
4   ((2 - beta) * prev_phase_val);
5 ap_fixed<32, 4, AP_RND> tmp_phase_pred =
6   tmp_first_part + tmp_sec_part - prev_prev_phase_val;
7 // To wrap around
8 if (tmp_phase_pred > M_PI) {
9     tmp_phase_pred = tmp_phase_pred - 2 * M_PI;
10 } else if (tmp_phase_pred < -M_PI) {

```

```

11     tmp_phase_pred = tmp_phase_pred + 2 * M_PI;
12 }
13 curr_phase_pred = tmp_phase_pred;

```

The complex calculations are split up to enable parallel execution and to reduce the critical path length. Additionally, the intermediate results are calculated with increased precision, namely `ap_fixed<32,4,AP_RND>`, 4 integer bits are needed to store the intermediate results before wrap around. The next stage is the “Update Model” stage, which updates the array values at the `counter` index accordingly, and in parallel, the conversion from polar to Cartesian form is performed. For conversion the functions `hls::sin(ap_fixed<16,3> phase)` and `hls::cos(ap_fixed<16,3> phase)` are used. The next step is to calculate the difference between the real measured and predicted values in Cartesian form. The resulting difference values are passed to two parallel working “Compress” blocks, which derive the region and the APPEND bits, and finally perform a look-up in the Huffman dictionary, which is stored in the BRAM. The Huffman code and the APPEND bits are packed by shifting the Huffman code to the left by `region`-amount of bits and performing a bit-wise OR with the APPEND bits. The code is shown below.

```

1 // pack the Huffman code and the APPEND bits
2 compressed_val = (huff_code << region) | append_bits;

```

The Huffman dictionary is an array of 16 *DictEntry_t* elements, which is a struct to store the Huffman code and the length of the code. It is shown below.

```

1 typedef struct {
2     ap_uint<14> val;
3     ap_uint<4> len; // max len 14
4 } DictEntry_t;

```

The parallel processed block’s final step is to pack the compressed real and imaginary data into a `ap_uint<58>` variable. The width of 58 was chosen to handle the worst case, when both Huffman encoded *S4* bits have the maximum length of 14 bits, and the APPEND bits both need 15 bits. The sum is $14 \times 2 + 15 \times 2 = 58$. The final step of the compression IP core is to combine the compressed data from each RX channel into a 256-bit wide output buffer. If one buffer is filled, it can be written to the output AXI Stream. With the last transaction to the output AXI Stream, the TLAST signal has to be triggered.

4.4 Challenges and Optimizations

This section describes the challenges encountered during the hardware implementation and the optimizations applied to address those challenges. The trade-offs in achieving a functional and efficient hardware implementation are highlighted. The focus of the implementation was to achieve a high throughput, including keeping the interval low to meet the real-time constraint and keeping the delay low to meet the constraints of a high target period.

4.4.1 Timing Closure

Because of long critical paths introduced by deep pipelines or complex arithmetic operations, the target clock period of 10 ns could not be met. The challenge was solved by adding intermediate variables and splitting complex arithmetic operations, and the target clock period was met.

4.4.2 Fixed-Point Precision

We used fixed-point precision to keep the resource usage in a reasonable range for such a complex algorithm. The intermediate calculations and intermediate results are performed and stored using a higher precision to minimize the effect of the lower precision. Creating a bit-accurate MATLAB simulation to get the same results as on the hardware is impossible. The different internal behavior in fixed-point arithmetic causes this.

4.4.3 Real-Time Performance Requirements

The throughput must be high to meet the real-time performance requirements and achieve high compatibility with various systems. Thus, the interval should be as low as possible. The aim was to achieve an interval of 1. The model prediction and compression must happen in parallel for each RX channel to achieve this. Firstly, this was achieved by complete partitioning of the model prediction arrays, using the pragma “ARRAY_PARTITION”, which is shown below for the `prev_mag_pred` array

```
1 #pragma HLS ARRAY_PARTITION variable = prev_mag_pred complete
```

This enables multiple load and store operations on the same array in one cycle. Secondly, the loop that should be unrolled is not allowed to have dependencies between the iterations, and thirdly, using the pragma “UNROLL”, which is shown below, the loop is unrolled.

```
1 #pragma HLS UNROLL
```

4.4.4 Resource Utilization

Resource utilization was not the focus of this work. However, it should still be kept within a reasonable range. To achieve that, fixed-point precision is used, and to reduce the utilization further, after having a fixed application with a fixed configuration, the needed performance could be calculated, and the overhead could be reduced by removing some parallel processing blocks, thus increasing the interval.

Chapter 5

Results

The purpose of this section is to present the key findings of the simulation and hardware implementation. Both results are compared to each other, and finally, the hardware implementation results are compared to the objectives and requirements introduced in Sections 1.2 and 3.3.

5.1 Simulation Results and Analysis

Section 3.2 describes the MATLAB simulation environment, and Section 3.4 describes the design space exploration with the results in Subsection 3.4.4. This section aims to summarize the main results of this design space exploration.

The performance metrics compression ratio, false negative detection, false positive detection, noise floor estimation, and signal-to-noise ratio are used to evaluate the various algorithms. Table 5.1 shows the simulation results of the algorithm defined as the design space.

Algorithm	CR	FN (%)	FP (%)	NF est. (dBFS)	SNR (dB)
FP16	1	0	0	-70.714	22.407
FX16	1	0.59	1.18	-70.691	22.33
CMPRA	2	0.59	0.59	-70.687	22.418
CMPRB	4	7.69	4.73	-70.253	22.111
EGE10	1.6	1.18	1.18	-70.689	22.403
EGE8	2	2.37	2.37	-70.64	22.289
EGE6	2.67	7.69	7.1	-70.367	22.197
EGE4	4	40.24	26.63	-74.689	28.245
DRHE	3.25	0.59	1.18	-70.691	22.33

Table 5.1: Design space exploration results using the MATLAB simulation environment

In Figure 5.1, the same results are shown as a scatter plot to visualize the performance of the various algorithms in a 2D space. In the scatter plot, we can see the trade-off between smaller size and lower data fidelity. The DRHE algorithm stands out in terms of a small size and still maintaining a high data quality.

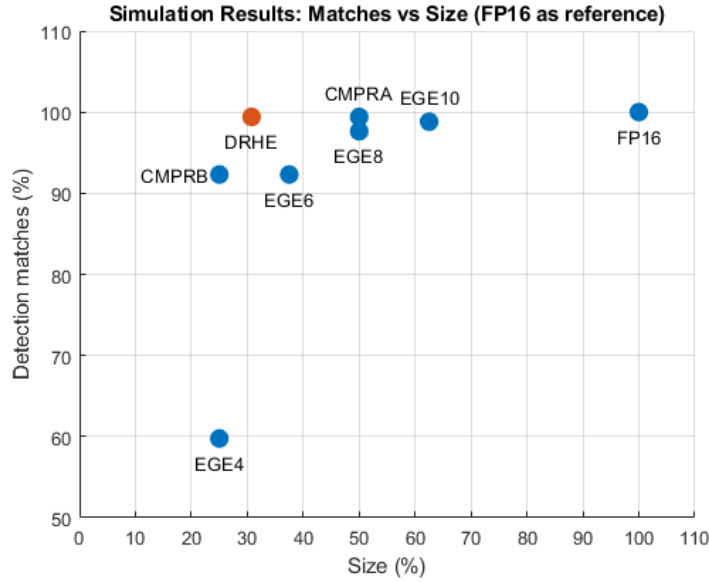


Figure 5.1: Scatter plot of the simulation results

Based on these results, the decision was made to take the DRHE algorithm as the basis for the final implementation.

5.2 Hardware Implementation: Results and Analysis

The purpose of this section is to present the results of the hardware implementation of the final compression algorithm, which was described in Section 3.5. The following subsections describe the evaluation setup and the resulting block design in Vivado. We evaluate the hardware implementation regarding processing speed, including throughput and latency, resource utilization, and, as a last point, we evaluate the resulting compressed data against the MATLAB simulation results.

5.2.1 Evaluation Setup

The evaluation setup is the same as the setup for the implementation. The used hardware is already introduced in Section 4.1. In Figure B.1, the used evaluation setup is shown with an enumeration of the used interfaces. The interface board is connected to the host system, the notebook, via the JTAG connector and the debug-UART port. Using the JTAG connection, the UltraScale+™ is programmed and can be debugged. For evaluation, the bare-metal application is loaded onto the ARM Cortex processor, which loads the input data from the SD card, stores it in memory, configures the IP cores via the AXI interface, and triggers the AXI DMA transactions. The compressed data is stored in the DDR memory, which is then printed to the serial output and onto a file on the SD card by the application.

The input data, which is stored on the SD card, is the preprocessed raw ADC data. The data is shifted to 16-bit full-scale, as shown in Subsection 3.2.1, meaning if the ADC

data has a bit width of 14, every sample is shifted to the left by $16 - 14 = 2$ bits. A common scale for all ADC data types is needed to fix the scaling schedule for the FFT IP core. The second preprocessing step is to apply the Hanning window with a length equal to the number of samples. The window is applied to each ramp. It reduces the effects of spectral leakage and improves the frequency resolution of the radar data.

5.2.2 Vivado Block Design

In Figure 5.2, the final system implementation block diagram is shown in the “Interface View”. The interface view shows only the interface level connectivity on the block design canvas. None of the other nets are shown when this view is selected, and if an IP core without any interfaces is present, this IP core will “disappear” from the block design. One IP core that disappeared is the “Constant” IP core, configuring the FFT cores’ scaling schedule with a constant value.

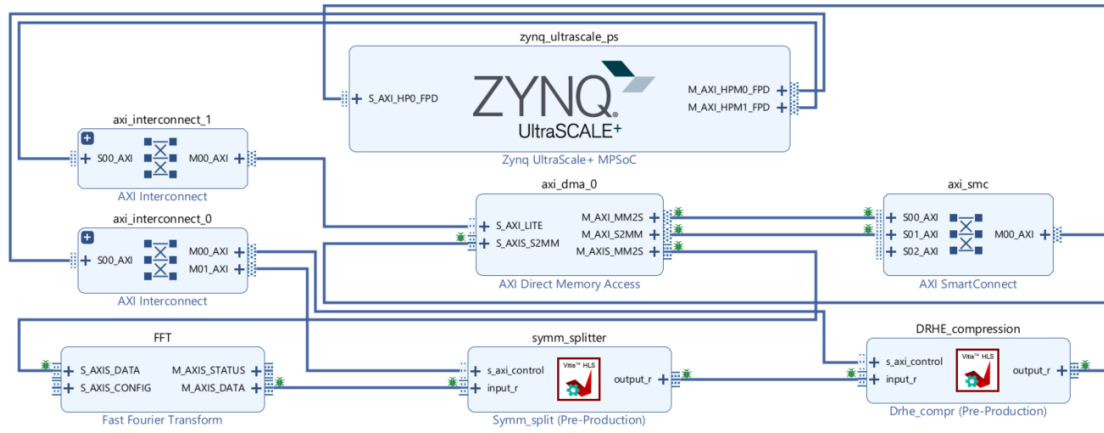


Figure 5.2: Final Block Design in Vivado

The IP core “`symm_splitter`” is inserted to reject the first half of the FFT output. Since we input a real-valued signal, the output consists of two symmetric halves. To be exact, the two halves are not exactly symmetric. The noise is more present in the lower frequency bins, and Xilinx therefore recommends using the upper half ($N/2 + 1$ to N points) of the output data when performing a real-valued FFT. The `symm_splitter` is configured with `nSamples` via the AXI4-Lite interface. It simply pipes the AXI Stream, but for the first half of transactions, it keeps `TVALID` low. In Figure 5.3, it can be clearly seen how the `symm_splitter` works.

5.2.3 Processing Speed

In this subsection, we focus on the processing speed with a focus on the latency and the throughput of the radar compression core. It is crucial to know and understand the processing speed in order to know if the real-time requirement is fulfilled. High throughput, low interval, and low latency ensure that the system can handle live radar data streams

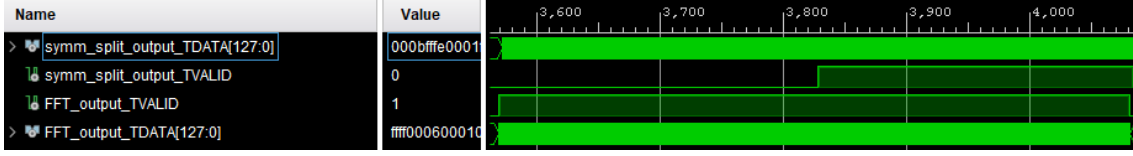


Figure 5.3: Waveform showing the working principle of the symmetry splitter IP core

without big delays. As we have already introduced, this is essential for safety-critical applications.

Throughput

Throughput is the amount of data processed per unit of time. In this section, we define it in data rate, in Gbit/s. Since the input and output AXI Stream widths are different, we define the throughput in terms of input and output throughput separately. The theoretical maximum throughput depends on the width of the input or output stream, the interval in which data is accepted or written to the stream, and the clock frequency. By reducing the critical path, the clock period of 10 ns is achieved in the final implementation of the compression IP core. That means the compression IP core can be driven with a clock frequency of 100 MHz. This work used a common clock for all the IP cores. The input throughput is defined by:

$$\begin{aligned} \text{Input throughput} &= \text{Input stream width} \times \text{Clock frequency} \\ &= 128 \times 100\,000\,000 = 12\,800\,000\,000 \text{ bits/sec} = 11.92 \text{ Gbit/s} \end{aligned} \quad (5.1)$$

The same parameters define the theoretical maximum output throughput as the input throughput:

$$\begin{aligned} \text{Output throughput} &= \text{Output stream width} \times \text{Clock frequency} \\ &= 256 \times 100\,000\,000 = 25\,600\,000\,000 \text{ bits/sec} = 23.84 \text{ Gbit/s} \end{aligned} \quad (5.2)$$

But even if the worst-case scenario were to happen, that all encoded *S4* bits have the maximum length of 14 bits and all the *APPEND* bits need 15 bits, we would reach per iteration for 4RX channels a maximum compressed output length of

$$\text{Output length} = (14 + 14 + 15 + 15) \times 4 = 232 \text{ bits}$$

So even with the worst-case performance with a negative compression from 128 bits to 232 bits, the output stream would not contain a transaction every clock cycle. The real maximum output throughput that considers the implementation is:

$$\begin{aligned} \text{Output throughput} &= \text{Output bits per cycle} \times \text{Clock frequency} \\ &= 232 \times 100\,000\,000 = 23\,200\,000\,000 \text{ bits/sec} = 21.61 \text{ Gbit/s} \end{aligned} \quad (5.3)$$

Since this is a compression IP core, we first of all don't want to reach this theoretical maximum output throughput because that would mean that the algorithm results in a negative compression. Second, since the compression algorithm has a variable-length output, the output throughput is not very useful for evaluation.

Latency

Latency refers to the time delay between when valid input data enters the system and when valid output data is produced. We evaluate the latency in three stages: The DRHE compression IP core as a standalone, the FFT IP core as a standalone, and the end-to-end latency measured in software.

The latency for the compression IP core as a standalone is already estimated during the development process by Vitis HLS. The estimation given is 23 cycles, or 230 ns. Another possibility is to measure the time from the first valid input to the first valid output using the Wave Viewer in Vivado. In Figure 5.4, the captured internal signals by the ILA core are shown in the waveform viewer. The latency is measured by measuring the time between the first time the TVALID signal is high on the input stream and the first time the TVALID signal is high on the output stream. The estimation and the real measurement differ because Vitis HLS does not measure when the first output transaction happens, so when the first 256 bits are filled with compressed data. In the real measurement, we measure when the first output transaction is made, and it takes 7 compressed input bins to fill up the complete 256-bit output buffer. So we get $23 + 7 = 31$ cycles.

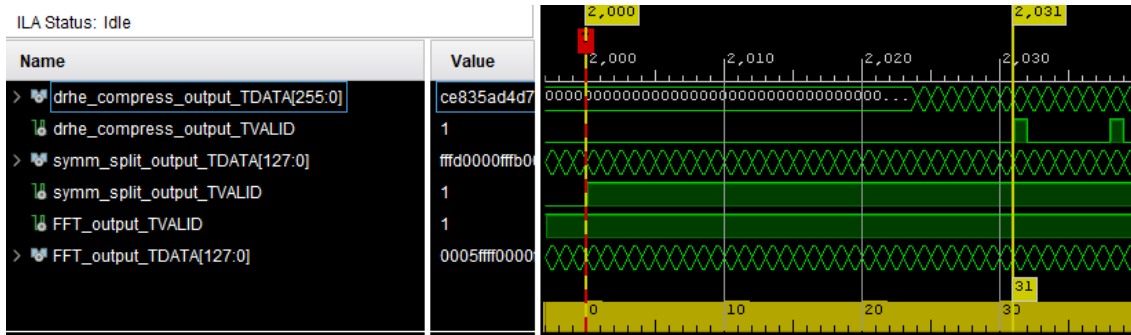


Figure 5.4: Latency measurement of the compression IP core using the waveform viewer

The latency for the FFT IP core as a standalone is measured the same way as for the compression IP core. In Figure 5.5, the latency measurement for the FFT core is shown in the waveform viewer. The measured latency is 1787 cycles, or 17 870 ns, while in Vivado, the reported latency by the FFT IP core is 1789 cycles.

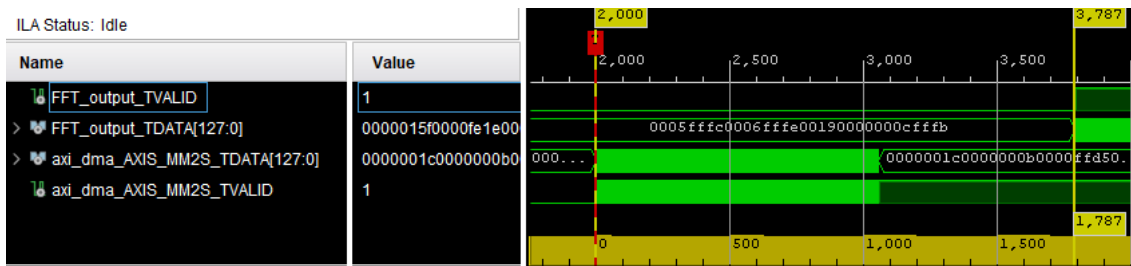


Figure 5.5: Latency measurement of the FFT IP core using the waveform viewer

The latency measurement for the system as a whole is measured as the time between receiving the TX interrupt and receiving the RX interrupt. The TX interrupt is triggered by the DMA IP core when it is finished reading the data from the memory, so with the last transfer from memory to the DMA. The RX interrupt is triggered when the **TLAST** signal is pulled high by the compression IP core or when the number of bytes specified is received. In our case, the interrupt is triggered by the **TLAST** signal. We chose an end-to-end latency measurement because of the availability of the interrupt signals. The measured latency is around 79 410 ns , or 0.079 ms.

5.2.4 Resource Utilization

This subsection presents the resource utilization of the radar data compression system as a whole and the compression IP core as a standalone. The resource usage is analyzed in terms of LUTs, look-up table random access memory (LUTRAM), FFs, DSP slices, Block RAM (BRAM), and global clock buffer (BUFG). In Figure 5.6, the implemented design is shown with leaf-cell highlighting of the **compression** IP core, the **FFT** IP core, and the **symm_splitter** IP core.

The exact numbers for the resource utilization of the evaluation system are listed in Table 5.2.

Resource	Utilization	Available	Utilization %
LUT	59353	70560	84.12
LUTRAM	2189	28800	7.60
FF	32552	141120	23.07
BRAM	38	216	17.59
DSP	80	360	22.22
BUFG	5	196	2.04

Table 5.2: Post-implementation resource utilization table for the system

Since the focus of this work is to implement the compression IP core, the numbers for the resource utilization for the **DRHE_compression** IP core are listed in Table 5.3.

Resource	Utilization	Available	Utilization %
LUT	45892	70560	65.04
LUTRAM	302	28800	1.05
FF	9243	141120	6.55
BRAM	10	216	4.63
DSP	44	360	12.22
BUFG	1	196	0.51

Table 5.3: Post-implementation resource utilization table for the compression IP core as a standalone

As we can see clearly in Table 5.2 and in Figure 5.6, a lot of resources are used by the compression IP core. The LUTs are typically used for implementing combinational logic but can also be used as RAM, called LUTRAM. 45 590 out of 70 560 LUTs are used for combinational logic by the DRHE compression IP core, which corresponds to 64.6 %. The

BRAM stores the arrays for model prediction and the fixed Huffman dictionary. DSP slices accelerate the arithmetic operations in the model prediction, and the design also utilizes 1 BUFG out of 196 available. BUFGs are specialized clock buffers that distribute clock signals across the FPGA fabric. They ensure low skew and high reliability for clock signals in timing-critical paths, making them essential for maintaining proper synchronization in the system.

Table 5.4 shows the resource utilization values for the `symm_splitter` IP core. Since the functionality is not very complex, the resources used are also low.

Resource	Utilization	Available	Utilization %
LUT	298	70560	0.42
LUTRAM	0	28800	0
FF	773	141120	0.55
BRAM	0	216	0
DSP	0	360	0
BUFG	0	196	0

Table 5.4: Post-implementation resource utilization table for the symmetry splitter IP core as a standalone

5.2.5 Evaluation Against Simulation Results

This subsection compares the performance and output to the reference MATLAB simulation. This comparison focuses on data fidelity differences, possibly caused by hardware-specific constraints, such as fixed-point arithmetic. Furthermore, it focuses on the compression performance differences that are caused by the hardware constraints.

Since we deal with a lossless compression scheme, with the limitation of being able to compress data in the 16-bit fixed-point format, we try to identify the impact of the fixed-point precision signal processing before the compression on data fidelity. Additionally, we identify the effect on the CR.

The CR of the HW implementation drops from 3.25 to 3.17, which is slightly lower compared to the CR of the MATLAB simulation of the DRHE algorithm. The simulation uses double precision for the processing steps before the compression, mainly consisting of windowing and FFT calculation. So, trying to identify the discrepancies between the simulation and the HW implementation, we quantized the windowed data to 16-bit fixed-point using:

```

1   wd_hanning = hanning(nSamples);
2   rawdata_wd = int16(double(rawdata) .* wd_hanning);

```

The resulting windowed and quantized raw data is stored on the SD card and used for hardware evaluation. Since the MATLAB FFT function and the FFT IP core do not deliver the same results, we used the bit-accurate C model offered with the Xilinx IP FFT core. For seamless MATLAB integration, we used the software MEX function, and using this model, all the configurations can be made precisely the same as in the hardware implementation. With these changes applied, the results show that the resulting range FFT data is now bit-accurate and all the performance metrics except CR match the hardware

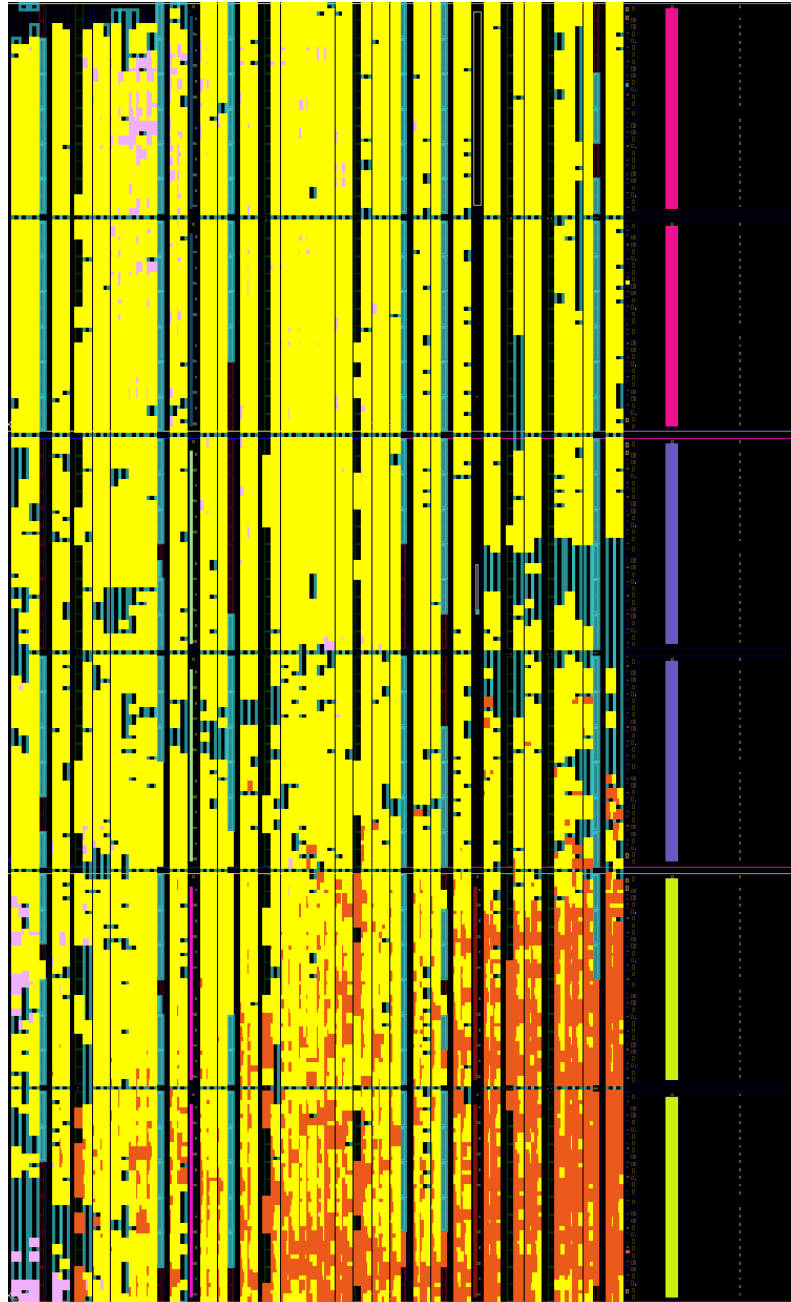


Figure 5.6: Implemented design in Vivado with highlighted leaf cells. Yellow is the DRHE_compression IP core, orange the FFT, and pink the symm_splitter IP core

implementation. The false negative detections increased from 0.59% to 1.18%, and the false positive detections increased from 1.18% to 2.37%. The noise floor was lowered, and the precision loss slightly increased the SNR. The CR still differs because creating a bit-accurate model prediction in MATLAB is impossible. So, to restore the compressed data from the HW implementation without any noise, we performed the decompression using a Vitis HLS testbench. In Vitis HLS, we developed a decompression routine with the bit-accurate inverse model prediction to map from difference data to range FFT data. In Table 5.5, the results from the design space exploration are listed with the DRHE bit-accurate simulation and the results from the hardware implementation.

Algorithm	CR	FN (%)	FP (%)	NF est. (dBFS)	SNR (dB)
FP16	1	0	0	-70.714	22.407
FX16	1	0.59	1.18	-70.691	22.33
DRHE	3.25	0.59	1.18	-70.691	22.33
DRHE (bit-accurate)	3.24	1.18	2.37	-71.751	23.303
HW impl	3.17	1.18	2.37	-71.751	23.303

Table 5.5: Design space exploration results using double precision for signal processing compared to the bit-accurate simulation and the hardware implementation (HW impl) results

5.3 Comparison to Requirements

The radar data compression system was designed to achieve several key objectives, including high compression efficiency, data fidelity, and real-time performance. The objectives are defined in Section 3.3. This section evaluates the hardware implementation results against these objectives.

Compression Ratio Requirement

The defined compression ratio requirement is 2.4 to transmit the data produced by the CTRX in the defined use case over a single Gigabit Ethernet cable. The CR of 2.4 considers only the radar data produced. If we also consider the headers that need to be transmitted, we can define the CR requirement as 2.7. The final implementation clearly fulfilled this requirement with an achieved CR of 3.17.

Real-Time Requirement

We also measured the introduced latency as an end-to-end measurement in software to simulate some transmission time. This type of measurement simulates a system with live data from a radar sensor since ramp by ramp is transmitted and can be compressed on-the-fly. So, measuring from transmitting the last raw ramp to receiving the last compressed transaction is the latency we are interested in. The defined real-time requirement of 15 ms is fulfilled with the measured latency of 0.08 ms. The compression IP core as a standalone introduces a latency of 230 ns or 0.00023 ms. The compression IP core achieves an input throughput of 11.92 Gbit/s, which is way higher than the maximum output data rate of

the CTRX. With the performance of the hardware implementation, it could be considered to optimize the implementation to use fewer resources and, in exchange, introduce more latency.

Data Fidelity Requirement

The overall goal was to keep the SNR as high as possible, and the target detection should match the reference data as closely as possible. With the compression scheme's limitation in compressing 16-bit fixed-point data, most of the information loss is caused. If the processing before the compression is performed using double floating-point precision, the detection performance and even the CR are slightly better than performing every block using fixed-point precision.

Chapter 6

Conclusion and Future Work

This thesis addressed the challenges arising from the vast amount of data generated by high-resolution radar sensors.

The proposed compression algorithm was initially developed and validated in MATLAB, where it was compared against various other compression techniques. The final algorithm was implemented on an AMD Zynq™ UltraScale+™ MPSoC, using fixed-point arithmetic, pipelining, and parallel processing. The final hardware implementation system integrated components such as an FFT core, the compression IP core, an AXI DMA for high-speed data transfer, and a bare-metal application for system control.

The system demonstrated strong performance of the compression IP core, achieving a compression ratio of 3.17 while still preserving the critical information of the data with 99.4 % detection matches with the reference data. Real-time performance requirements were exceeded, with the system achieving an input throughput of 11.92 Gbit/s and an end-to-end latency of 0.08 ms. Resource utilization remained within acceptable limits, with 65 % of LUTs, 12.2 % of DSP slices, and 4.6 % of BRAM utilized, leaving sufficient room for potential future enhancements.

Despite these successes, certain limitations were identified. By using 16-bit fixed-point quantization, a minor precision loss was introduced, affecting the data fidelity. Additional precision loss was introduced by using fixed-point arithmetic in the processing before the compression IP core. To make the scheme useful in practice, safe error handling has to be added, and depending on the application several adaptations and extensions have to be made, such as a configurable predefined compression ratio, with potential data dropping if not reached, or compressing over a range bin to transpose the data for a faster doppler-FFT.

In conclusion, this thesis demonstrates the feasibility of implementing an efficient hardware-accelerated radar data compression system for automotive applications. By reducing bandwidth requirements without sacrificing critical radar data, the proposed system provides a scalable and efficient solution, paving the way for its integration into future ADAS and autonomous driving technologies.

6.1 Future Work

This section discusses potential directions for future research and development, focusing on addressing the current system's limitations and exploring new opportunities for improvement and integration.

Addressing Fixed-Point Precision Limitations

The current system utilizes 16-bit fixed-point arithmetic to optimize resource usage on the FPGA. While effective, the processing chain before the compression algorithm introduces a major precision loss using this approach. Since the compression IP core greatly exceeds the real-time performance requirement, the processing speed performance could be lowered to save resources. Instead, the impact of using floating-point arithmetic for the FFT and the model prediction could be investigated.

Bit-Accurate Decompression C-Model

The focus of this work was to develop and implement a compression algorithm. The next step is to investigate the decompression performance. A bit-accurate and efficient C-model should be developed to be used and executed on every system the radar system connects to.

Real-World System Testing

Future work should focus on integrating the compression IP core into a system with a real radar sensor. The algorithm should be tested extensively under various conditions and with all possible configuration options to see its effectiveness in multiple situations.

Transpose Compression

As introduced in Subsection 2.2, the Doppler-FFT operation is performed on one range bin over all the ramps. If the data is stored in compressed format in memory, all the data needs to be decompressed to be able to perform the first Doppler-FFT. Even if the data is decompressed on-the-fly and stored in decompressed format in memory, there are two ways to handle the data. Either during decompression, the data is written in transposed order to the memory, meaning that the range and Doppler dimensions are exchanged, or the data is stored in the original order to the memory, and to perform the Doppler-FFT for each bin, a separate memory location needs to be loaded. Compression can be performed over the ramps to fix this problem. So the range bin 1 of ramp 1 is compressed and stored in `buffer1`, range bin 2 of ramp 1 is compressed and stored in `buffer2`, and so on until ramp 2 is received. Range bin 1 of ramp 2 is compressed and aligned with the data in `buffer1`, and the compressed range bin 2 is stored in `buffer2`. This is continued until the frame is completely compressed. Buffering is introduced using this approach; thus, more resources are used, and the latency will also increase.

Appendix A

Fixed Huffman Dictionary

S4 Value	S4 - Decimal	Huffman Code	Length
0000	0	1010	4
0001	1	100	3
0010	2	01	2
0011	3	00	2
0100	4	11	2
0101	5	10110	5
0110	6	101110	6
0111	7	1011110	7
1000	8	10111110	8
1001	9	101111110	9
1010	10	1011111110	10
1011	11	10111111110	11
1100	12	10111111111111	14
1101	13	101111111111110	14
1110	14	101111111111110	13
1111	15	1011111111110	12

Table A.1: Fixed Huffman dictionary for S4 values

Appendix B

Evaluation Hardware Setup

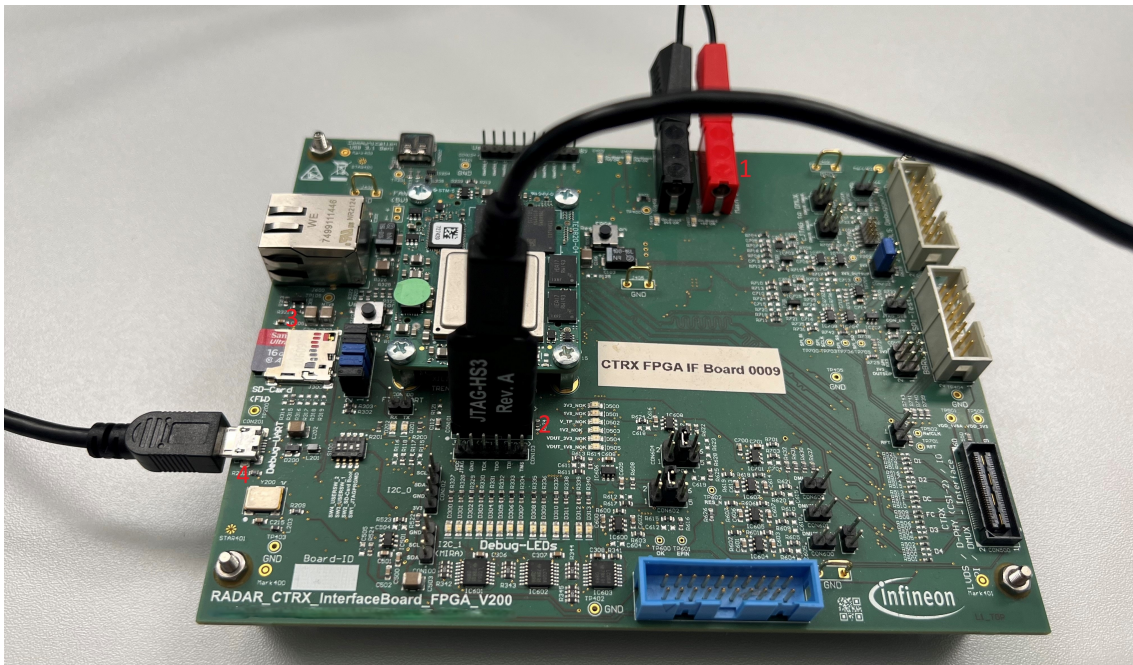


Figure B.1: Evaluation Setup with the used connections. (1) Power supply (2) JTAG-connection to the host system (3) SD-card module (4) Debug-UART connection to the host system

Appendix C

Acronyms

ABS Anti-lock Braking System

ACC Adaptive Cruise Control

ADAS Advanced Driver Assistance Systems

AEB Automatic Emergency Brake

ARM Advanced RISC Machines

ASIC Application-Specific Integrated Circuit

AWGN Additive White Gaussian Noise

AXI Advanced eXtensible Interface

BRAM Block Random Access Memory

BSD Blind Spot Detection

CLB Configurable Logic Blocks

CR Compression Ratio

CW Continuous Wave

DCT Discrete Cosine Transform

DMA Direct Memory Access

DWT Discrete Wavelet Transform

DSP Digital Signal Processing

ECU Electronic Control Unit

EGE Exponential Golomb Encoding

FF Flip-Flops

FFT	Fast Fourier Transform
FMC	FPGA Mezzanine Card
FMCW	Frequency Modulated Continuous Wave
FN	False Negative
FP	False Positive
HDL	Hardware Description Language
HLS	High-Level Synthesis
IF	Intermediate Frequency
IFX	Infineon Technologies AG
ILA	Integrated Logic Analyzer
IP	Intellectual Property
LCA	Lane Change Assist
LPC	Linear Predictive Coding
LSB	Least Significant Bits
LUT	Look-Up Table
LUTRAM	Look-Up Table Random Access Memory
MCU	Micro Controller Unit
MMIC	Monolithic Microwave Integrated Circuit
MPSoC	Multi-Processor System-on-Chip
NCI	Non-Coherent Integration
OSR	Output Sample Rate
PL	Programmable Logic
PS	Processing System
RADAR	Radio Detection And Ranging
RMS	Root Mean Square
RLE	Run-Length Encoding
RTL	Register Transfer Level
SAR	Synthetic Aperture Radar

SDV Software Defined Vehicle

SNR Signal-to-Noise Ratio

SoM System-on-Module

TI Texas Instruments

Bibliography

- [1] David A. Huffman. “A Method for the Construction of Minimum-Redundancy Codes”. In: *Proceedings of the IRE* 40.9 (1952), pp. 1098–1101. DOI: 10.1109/JRPROC.1952.273898.
- [2] Jukka Teuhola. “A compression method for clustered bit-vectors”. In: *Information Processing Letters* 7.6 (1978), pp. 308–311. ISSN: 0020-0190. DOI: [https://doi.org/10.1016/0020-0190\(78\)90024-8](https://doi.org/10.1016/0020-0190(78)90024-8). URL: <https://www.sciencedirect.com/science/article/pii/0020019078900248>.
- [3] J. Rissanen and G. G. Langdon. “Arithmetic Coding”. In: *IBM Journal of Research and Development* 23.2 (1979), pp. 149–162. DOI: 10.1147/rd.232.0149.
- [4] “IEEE Standard for Digitizing Waveform Recorders”. In: *IEEE Std 1057-1994* (1994). DOI: 10.1109/IEEESTD.1994.122649.
- [5] U. Benz, K. Strod, and A. Moreira. “A comparison of several algorithms for SAR raw data compression”. In: *IEEE Transactions on Geoscience and Remote Sensing* 33.5 (1995), pp. 1266–1276. DOI: 10.1109/36.469491.
- [6] M. Frigo and S.G. Johnson. “FFTW: an adaptive software architecture for the FFT”. In: *Proceedings of the 1998 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP '98 (Cat. No.98CH36181)*. Vol. 3. 1998, 1381–1384 vol.3. DOI: 10.1109/ICASSP.1998.681704.
- [7] Steven Pigeon. “Huffman coding”. In: *Lossless Compression Handbook* (2003), pp. 79–100.
- [8] Graham M Brooker et al. “Understanding millimetre wave FMCW radars”. In: *1st international Conference on Sensing Technology*. Vol. 1. 2005.
- [9] Eric Monmasson and Marcian N. Cirstea. “FPGA Design Methodology for Industrial Control Systems—A Review”. In: *IEEE Transactions on Industrial Electronics* 54.4 (2007), pp. 1824–1842. DOI: 10.1109/TIE.2007.898281.
- [10] M.I. Skolnik. *Radar Handbook, Third Edition*. Electronics electrical engineering. McGraw-Hill Education, 2008. ISBN: 9780071485470. URL: <https://books.google.at/books?id=76uF2Xebm-gC>.
- [11] OMAR MAHDI, Mazin Mohammed, and Ahmed Mohamed. “Implementing a Novel Approach an Convert Audio Compression to Text Coding via Hybrid Technique”. In: (Nov. 2013).

- [12] Liang Li. *Embedded Data Compression in Automotive FMCW Radar System*. [Online; accessed 03-April-2025]. 2014. URL: <https://essay.utwente.nl/65925/1/thesis.pdf>.
- [13] Zoran Zivkovic and Liang Li. “On-the-fly range-velocity radar data compression exploring Doppler spectrum redundancy”. In: *2015 16th International Radar Symposium (IRS)*. 2015, pp. 321–326. DOI: 10.1109/IRS.2015.7226292.
- [14] Ruchi Gupta, Mukesh Kumar, and Rohit Bathla. “Data compression-lossless and lossy techniques”. In: *International Journal of Application or Innovation in Engineering & Management* 5.7 (2016), pp. 120–125.
- [15] IEEE. *IEEE P802.3bp 1000BASE-T1 PHY Task Force*. [Online; accessed 20-March-2025]. 2016. URL: <https://www.ieee802.org/3/bp/>.
- [16] P Kavitha. “A survey on lossless and lossy data compression methods”. In: *International Journal of Computer Science & Engineering Technology* 7.03 (2016), pp. 110–114.
- [17] Ke Ma, Chongyang Li, and Haozheng Li. “A new hybrid data compression algorithm for weather radar”. In: *2016 11th International Symposium on Antennas, Propagation and EM Theory (ISAPE)*. 2016, pp. 886–889. DOI: 10.1109/ISAPE.2016.7834103.
- [18] Qiangyu Zeng et al. “Weather Radar Data Compression Based on Spatial and Temporal Prediction”. In: *Atmosphere* 9.3 (2018). ISSN: 2073-4433. DOI: 10.3390/atmos9030096. URL: <https://www.mdpi.com/2073-4433/9/3/96>.
- [19] Anil Mani, Zigang Yang. *Memory Compression and Decompression Engine for T1 mmWave Radar*. [Online; accessed 03-April-2025]. 2019. URL: <https://www.ti.com/lit/an/swra663/swra663.pdf>.
- [20] Victor Bandur et al. “Making the Case for Centralized Automotive E/E Architectures”. In: *IEEE Transactions on Vehicular Technology* 70.2 (2021), pp. 1230–1245. DOI: 10.1109/TVT.2021.3054934.
- [21] Kirsten Matheus and Thomas Königseder. *Automotive ethernet*. Cambridge University Press, 2021.
- [22] V. B. Polyakov, N. A. Ignatova, and A. A. Sentsov. “Multi-Criteria Selection of the Radar Data Compression Method”. In: *2021 Wave Electronics and its Application in Information and Telecommunication Systems (WECONF)*. 2021, pp. 1–4. DOI: 10.1109/WECONF51603.2021.9470755.
- [23] Christian Waldschmidt, Juergen Hasch, and Wolfgang Menzel. “Automotive Radar — From First Efforts to Future Systems”. In: *IEEE Journal of Microwaves* 1.1 (2021), pp. 135–148. DOI: 10.1109/JMW.2020.3033616.
- [24] Zishuai Wang et al. “Data Compression and Transmission Technology of USV Radar Image”. In: *2021 IEEE International Conference on Unmanned Systems (ICUS)*. 2021, pp. 644–647. DOI: 10.1109/ICUS52573.2021.9641397.
- [25] Francesco Mancuso Giulio Meucci. *FMCW radar data compression*. [Online; accessed 4-April-2025]. 2022. URL: https://github.com/mandugo/FMCW-Radar-Data-Compression/blob/main/FMCW_radar_data_compression.pdf.

- [26] Zongwei Liu, Wang Zhang, and Fuquan Zhao. “Impact, challenges and prospect of software-defined vehicles”. In: *Automotive Innovation* 5.2 (2022), pp. 180–194.
- [27] Yongxin Shan et al. “Implementation of Fast Huffman Encoding Based on FPGA”. In: *Journal of Physics: Conference Series* 2189.1 (Feb. 2022), p. 012021. DOI: 10.1088/1742-6596/2189/1/012021. URL: <https://dx.doi.org/10.1088/1742-6596/2189/1/012021>.
- [28] cadence. *What Is Zonal Architecture? And Why Is it Upending the Automotive Supply Chain?* [Online; accessed 27-March-2025]. 2023. URL: https://community.cadence.com/cadence_blogs_8/b/breakfast-bytes/posts/zonal.
- [29] The MathWorks Inc. *MATLAB Version: 23.2.0.2365128 (R2023b)*. Natick, Massachusetts, United States, 2023. URL: <https://www.mathworks.com>.
- [30] Shanliang Yao et al. “Exploring Radar Data Representations in Autonomous Driving: A Comprehensive Review”. In: 2023. URL: <https://api.semanticscholar.org/CorpusID:266149576>.
- [31] Infineon Technologies AG Market News. *Infineon’s new radar MMIC RASIC™ CTRX8191F enables the next generation of 4D and HD imaging radars*. <https://www.infineon.com/cms/en/about-infineon/press/market-news/2024/INFATV202412-035.html>. Online; accessed 24-March-2025. 2024.
- [32] SemiconductorEngineering. *Automotive Architectures*. [Online; accessed 27-March-2025]. 2024. URL: https://semiengineering.com/knowledge_centers/automotive/automotive-architectures/.
- [33] Thomas Böhm. *RISC-V – Success factors & opportunities for dependable automotive applications*. [Online; accessed 28-March-2025]. 2024. URL: <https://riscv-europe.org/summit/2024/media/proceedings/plenary/Wed-09-00-Thomas-Boehm.pdf>.
- [34] Inc. AMD Advanced Micro Devices. *Zynq™ UltraScale+™ MPSoC Data Sheet*. [Online; accessed 17-April-2025]. URL: <https://docs.amd.com/v/u/en-US/ds891-zynq-ultrascale-plus-overview>.
- [35] Infineon Technologies AG. *Radar Sensors*. <https://www.infineon.com/cms/en/product/sensor/radar-sensors/>. Online; accessed 24-March-2025.
- [36] Trenz Electronic. *TE0820 TRM*. [Online; accessed 17-April-2025]. URL: <https://wiki.trenz-electronic.de/display/PD/TE0820+TRM>.
- [37] Lohit M. Kadlaskar Jagadish H. Pujar. “A NEW LOSSLESS METHOD OF IMAGE COMPRESSION AND DECOMPRESSION USING HUFFMAN CODING TECHNIQUES”. In: *Journal of Theoretical and Applied Information Technology* ().
- [38] Synopsys. *What is ADAS (Advanced Driver Assistance Systems)?* <https://www.synopsys.com/glossary/what-is-adas.html>. Online; accessed 24-March-2025.